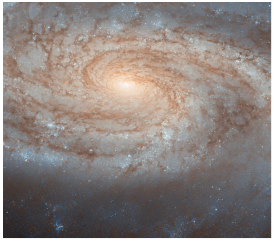
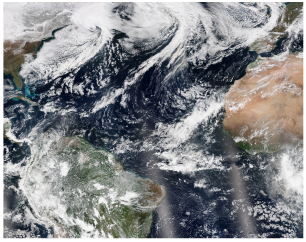
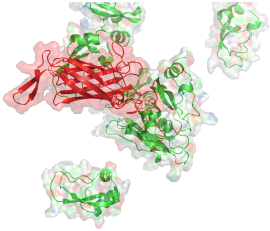
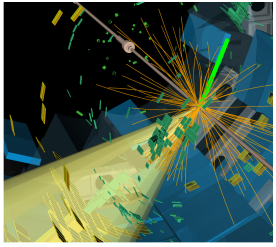





High performance scientific computing in C++

HPC C++ Course 2026

12 October – 15 October 2026 | Sandipan Mohanty | Forschungszentrum Jülich, Germany

Member of the Helmholtz Association

JÜLICH Forschungszentrum

Chapter 2

Cost of ...

JÜLICH Forschungszentrum

```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

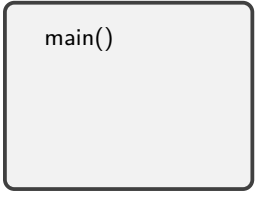
```
auto h211(int i)
    -> int
{
    return -i;
}
```

```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

```
auto h211(int i)
    -> int
{
    return -i;
}
```

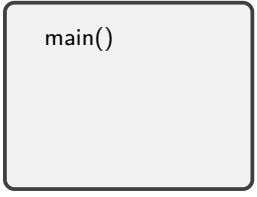


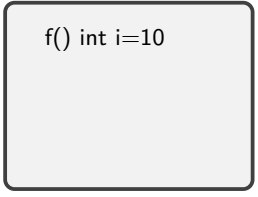
```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

```
auto h211(int i)
    -> int
{
    return -i;
}
```



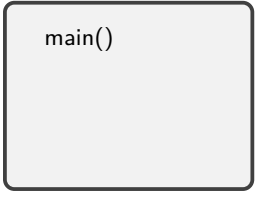


```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

```
auto h211(int i)
    -> int
{
    return -i;
}
```



```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

```
auto h211(int i)
    -> int
{
    return -i;
}
```

main()

g() int i = 10

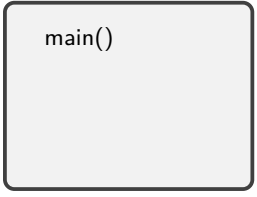
JÜLICH Forschungszentrum

```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

```
auto h211(int i)
    -> int
{
    return -i;
}
```



```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

```
auto h211(int i)
    -> int
{
    return -i;
}
```

main()

h1() int i = 10

JÜLICH Forschungszentrum

```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

```
auto h211(int i)
    -> int
{
    return -i;
}
```

main()

h1() int i = 10

h11() int i = 10

JÜLICH Forschungszentrum

```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

```
auto h211(int i)
    -> int
{
    return -i;
}
```

main()

h1() int i = 10

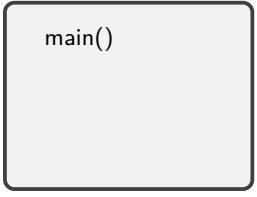
JÜLICH Forschungszentrum

```
auto main() -> int
{
    auto N = 10;
    if (f(N) < g(N)) {
        h1(N);
    } else {
        h2(N);
    }
}
```

```
auto f(int i)() -> int
{
    return (i * i) %12;
}
auto g(int i)() -> int
{
    return i % 12;
}
auto h1(int i)() -> int
{
    return h11(i);
}
 auto h2(int i)() -> int
{
    return h21(i);
}
```

```
auto h11(int i)() -> int
{
    return i * i;
}
auto h21(int i)() -> int
{
    return i + h211(i);
}
```

```
auto h211(int i)
    -> int
{
    return -i;
}
```



Stack

```
1  class V3 {
2      double x{}, y{}, z{};
3      auto cross(const V3 &) -> V3;
4      auto dot(const V3 &) -> double;
5  };
6  auto prob(int i, const V3& x, const V3& y)
7      -> double
8  {
9      int j = i % 233;
10     V3 tmp{x};
11     for (; j < i; ++j) {
12         tmp = tmp.cross(y);
13     }
14     return tmp.dot(x);
15 }
```

- Heavily reused memory locations
- Likely cached, therefore, fast
- All local (block scope) variables of any type, which come into existence inside a block, and expire at the end of the block, i.e., with *automatically managed* lifetime.

Global storage

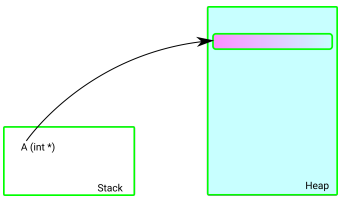
```
1  auto prob(int i) -> double
2  {
3      static int c{0};
4      ++c;
5      if (c % 1000==0) {
6          std::cout << "Call count reached "
7            << c << "\n";
8      }
9      static const double L[] = {3.14, 2.71};
10     return L[i % 2];
11 }
```

- Variables outside any function
- Variables marked with the `static` keyword in functions
- Floating point constants, array initializer lists, jump tables, virtual function tables

JÜLICH Forschungszentrum

Heap

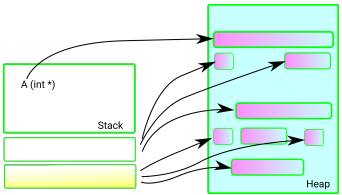
```
1  void f()
2  {
3      int *A = new int[1000000];
4      // calculations with A
5      delete [] A;
6  }
```



- Directly/indirectly managed memory through **`new`**, **`delete`**, `malloc` or `free`
- Best practice: managed by container types like `vector`, `list` etc. or by smart pointers `unique_ptr` or `shared_ptr`
- Objects who come into existence with a **`new`** call, and live until an explicit **`delete`** call
- Can store very large objects which don't fit in the stack
- Arrays whose size is not known at compile time. C99 style variable length arrays are not standard C++.

Heap

```
1  void f()
2  {
3      int *A = new int[1000000];
4      // calculations with A
5      // What if we throw an exception here
6      // and never actually reach the delete?
7      delete [] A;
8  }
```



- Must remember to free memory before all pointers pointing to that heap block go out of scope. Those pointers may expire either because the program successfully runs past the `}` marking the end of their lifetime, or leaves the scope by throwing an exception. $\Longrightarrow$ RAII: tie the acquiring and releasing of resources to the life time of a suitable object.
- Tends to get fragmented
- Must find a suitably sized unused block, and must keep track of what is and isn't in use $\longmapsto$ allocation and deallocation are expensive
- Objects stored one after the other may end up in very different locations
- Slower than stack storage

Exercise 3.1:

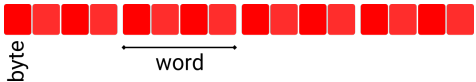
In HPC, we have to carefully monitor our heap allocation/deallocation operations. In the program `examples/alloc_cost.cc`, we compare two nearly identical functions, where the only difference is the use of a heap allocated array as the returned value. We clearly see that the version without the heap allocation runs faster. Reducing the allocation/deallocation operations inside hot code sections improves performance.

```cpp
static void StringCreation(benchmark::State& state) {
  for (auto _ : state) {
    std::string created_string("hello");
    benchmark::DoNotOptimize(created_string);
  }
}
BENCHMARK(StringCreation);
static void StringCopy(benchmark::State& state) {
  std::string x = "hello";
  for (auto _ : state) {
    std::string copy(x);
  }
}
BENCHMARK(StringCopy);
```

Exercise 3.2:

Test the cost of allocation and deallocation using the microbenchmarking site quick-bench.com! Their default example is the code given here (above), comparing string creation and copy. Note down the timings. Then add about 20 'o's at the end of the "hello" in each bench mark, i.e., "hello" $\longmapsto$ "hellooooooooooooooooooooo". Compare the timings again! Reduce the number of o's until the timings are as in the original form. Do you understand the timings?

Data alignment



- Data is read or written with a unit size called word. On the most common architectures, word size is 4 or 8 bytes.
- Data alignment means, putting data on memory addresses which are integral multiples of the word size
- n-byte aligned address has $>= log_2(n)$ least significant zeros
- Access for aligned data is fast
- If the size of a primitive type does not exceed the word size, access to aligned data of that type is also atomic

Data alignment

- The X86 architecture is tolerant of misaligned data. Programs run, even if they can't use SSE features
- PowerPC throws a hardware exception, which may be handled by the OS. For unaligned 8 byte access, a 4,610% performance penalty has been discussed (`http://www.ibm.com/developerworks/library/pa-dalign/`)
- On other systems, crashes, data corruption, incorrect results are all possibilities

Data alignment

- Usually, primitive types are aligned by their "natural alignment": 4 byte `int` has 4 byte alignment, 8 byte double has alignment of 8 and so on
- A class has a natural alignment equal to the strictest requirement of its members
- The `alignof` operator can be used to query the alignment of a type
- The `alignas` keyword can be used to set a stricter alignment requirement

Exercise 3.3:

Verify the above using the example program `examples/alignof.cc`.

Data structure padding

```
class D { // alignment : 8, because of d
    int nm; // alignment requirement 4.
    double d; // Must have alignment 8.
public:
    void val(double x) { d=x; }
    auto val() const -> double { return d; }
    auto operator+(double x1) const -> double;
auto D::operator+(double x) const -> double
{
    return d + x * x;
D::operator+(double) const:
    vfmadd213sd     xmm0, xmm0, QWORD PTR [rdi+8]
    ret
```

- Alignment requirement of members can necessitate introduction of padding between members

JÜLICH Forschungszentrum

Data structure padding

```
class D { // alignment : 8, because of d
    int nm; // alignment requirement 4.
    double d; // Must have alignment 8.
public:
    void val(double x) { d=x; }
    auto val() const -> double { return d; }
    auto operator+(double x1) const -> double;
auto D::operator+(double x) const -> double
{
    return d + x * x;
D::operator+(double) const:
    vfmadd213sd     xmm0, xmm0, QWORD PTR [rdi+8]
    ret
```

- Alignment requirement of members can necessitate introduction of padding between members
- What happens to the assembler here, if we put a comma between n and m in the name `nm` in class `D`?

JÜLICH Forschungszentrum

Data structure padding

```
class D { // alignment : 8, because of d
    int nm; // alignment requirement 4.
    double d; // Must have alignment 8.
public:
    void val(double x) { d=x; }
    auto val() const -> double { return d; }
    auto operator+(double x1) const -> double;
auto D::operator+(double x) const -> double
{
    return d + x * x;
D::operator+(double) const:
    vfmadd213sd     xmm0, xmm0, QWORD PTR [rdi+8]
    ret
```

- Alignment requirement of members can necessitate introduction of padding between members
- What happens to the assembler here, if we put a comma between n and m in the name `nm` in class `D`?
- What if we make it `int n, m, p;`? Test it using the compiler explorer! Click on the link or copy and paste code from `examples/assembly/class2.cc`.

JÜLICH Forschungszentrum

Data structure padding

```
1  class A {
2      char c;
3      double x;
4      int d;
5  };
6  // Compiled as if it was ...
7  char c;
8  char pad[7];
9  double x;
10 int d;
11 char pad2[4]; // why is this here ?
12 // Overall alignment alignof(double)
13 // size of struct = 24
```

```
1  class B {
2      double x;
3      int d;
4      char c;
5  };
6  // Compiled as if it was ...
7  double x;
8  int d;
9  char c;
10 char pad[3];
11 // Overall alignment alignof(double)
12 // size of struct = 16
```

- Due to padding, size of structures can be bigger than the sum of sizes of their elements
- C++ rules do not allow the compiler to reorder elements for space
- Carefully choosing the declaration order of class members can save memory

Alignment specifiers

```
1  alignas(64) double x[4]; // ok
2
3  alignas(64) vector<double> a(4);
4  // Pointless.
5  // The above simply aligns the resource
6  // handle, not the data on the heap
7
8  alignas(64) array<double, 4> A;
9  // This is fine, as std::array has
10 // real data in its struct
11
12 template <class T, int vecsize>
13 struct alignas(vecsize) simd_t
14 {
15     array<T,vecsize/sizeof(T)> data;
16 };
17 // We have requested that all objects
18 // of type simd_t should be aligned
19 // to vecsize bytes.
```

- The `alignas` keyword can specify alignment for variables
- Can be attached to a class declaration so that all objects of that type have a specified alignment
- It is possible to attach an extended alignment specifier to the class declaration
- Be mindful about what you are aligning when you use `alignas` for a resource handle like `vector`

```
1  alignas(64) std::vector U(100UL, 3.14);
2  // Align the vector object on the stack
3  // The array managed by the vector is
4  // not aligned
5  std::vector<double,
6      tbb::cache_aligned_allocator<double>>
7          A(100UL, 3.14);
8  // Cache aligned data array on the heap
```

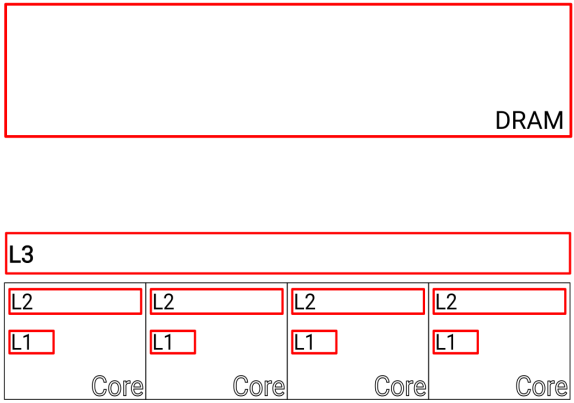
Exercise 3.4:

The `examples/align0.cc` has an example class template, which creates a data array of the right size to fill the simd vector width irrespective of the input data type. It illustrates the use of **`alignof`** and **`alignas`**, and variable templates.

Exercise 3.5:

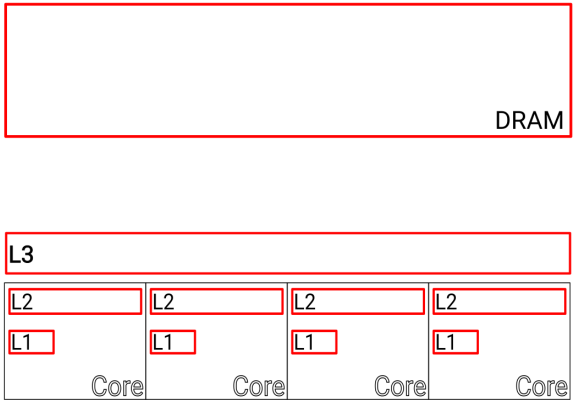
The `examples/align1.cc` shows that the usual mechanisms of dynamic allocation up to C++14 do not provide any guarantees about alignment greater than the natural alignment of the type. The behaviour changed in C++17 for types with explicitly specified extended alignment specifier like our `simd_t` class of the previous example. Finally, `examples/align2.cc` shows the use of a new version of the **`new`** operator introduced in C++17, which accepts an alignment argument.

Memory



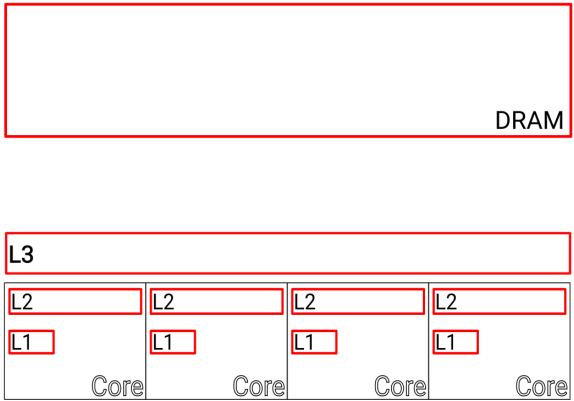
- Fun fact: in 1 clock cycle of the CPU on my laptop, a photon travels about 10 cm in vacuum!

Memory



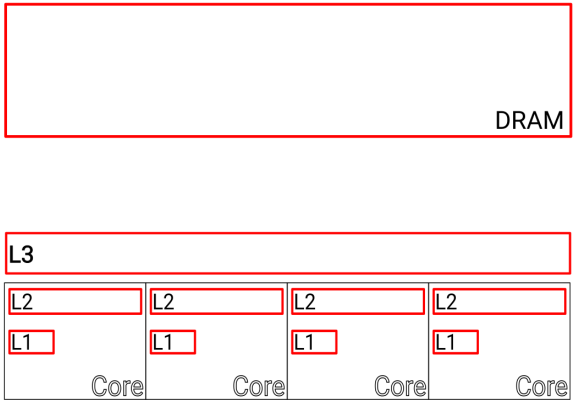
- Fun fact: in 1 clock cycle of the CPU on my laptop, a photon travels about 10 cm in vacuum!
- CPUs contain a certain amount of “cache” memory, which is faster to access, but much smaller than RAM
- Cost of fetching one integer from the main memory can be a hundred times larger than getting it from the L1 cache
- When the CPU looks for data from one address in memory, it is copied from RAM to the cache and then used.

Memory



- Fun fact: in 1 clock cycle of the CPU on my laptop, a photon travels about 10 cm in vacuum!
- CPUs contain a certain amount of “cache” memory, which is faster to access, but much smaller than RAM
- Cost of fetching one integer from the main memory can be a hundred times larger than getting it from the L1 cache
- When the CPU looks for data from one address in memory, it is copied from RAM to the cache and then used.

Memory



- Fun fact: in 1 clock cycle of the CPU on my laptop, a photon travels about 10 cm in vacuum!
- CPUs contain a certain amount of "cache" memory, which is faster to access, but much smaller than RAM
- Cost of fetching one integer from the main memory can be a hundred times larger than getting it from the L1 cache
- When the CPU looks for data from one address in memory, it is copied from RAM to the cache and then used.
- If it is immediately accessed again, it is in the cache, and can be used without the cost of fetching it from RAM
- Memory is fetched in "cache lines". Successive operations on contiguous memory locations do not incur the full cost of main memory access

Finding out cache information about your CPU

```
root > dmidecode -t cache
# dmidecode 3.1
Getting SMBIOS data from sysfs.
SMBIOS 3.0.0 present.
Handle 0x0007, DMI type 7, 19 bytes
Cache Information
        Socket Designation: L1 Cache
        Configuration: Enabled, Not Socketed, Level 1
        Operational Mode: Write Back
        Location: Internal
        Installed Size: 256 kB
        Maximum Size: 256 kB
        Supported SRAM Types:
                Synchronous
        Installed SRAM Type: Synchronous
        Speed: Unknown
        Error Correction Type: Parity
        System Type: Unified
        Associativity: 8-way Set-associative}

Handle 0x0008, DMI type 7, 19 bytes
Cache Information
        Socket Designation: L2 Cache
        Configuration: Enabled, Not Socketed, Level 2
        Operational Mode: Write Back
        Location: Internal
        Installed Size: 1024 kB
        Maximum Size: 1024 kB

        Supported SRAM Types:
                Synchronous
        Installed SRAM Type: Synchronous
        Speed: Unknown
        Error Correction Type: Single-bit ECC
        System Type: Unified
        Associativity: 4-way Set-associative
Handle 0x0009, DMI type 7, 19 bytes
Cache Information
        Socket Designation: L3 Cache
        Configuration: Enabled, Not Socketed, Level 3
        Operational Mode: Write Back
        Location: Internal
        Installed Size: 8192 kB
        Maximum Size: 8192 kB
        Supported SRAM Types:
                Synchronous
        Installed SRAM Type: Synchronous
        Speed: Unknown
        Error Correction Type: Multi-bit ECC
        System Type: Unified
        Associativity: 16-way Set-associative
```


JÜLICH Forschungszentrum

CPU cache

```
not_root> getconf -a | grep -i cache
LEVEL1_ICACHE_SIZE                 32768
LEVEL1_ICACHE_ASSOC                8
LEVEL1_ICACHE_LINESIZE             64
LEVEL1_DCACHE_SIZE                 32768
LEVEL1_DCACHE_ASSOC                8
LEVEL1_DCACHE_LINESIZE             64
LEVEL2_CACHE_SIZE                  262144
LEVEL2_CACHE_ASSOC                 4
LEVEL2_CACHE_LINESIZE              64
LEVEL3_CACHE_SIZE                  8388608
LEVEL3_CACHE_ASSOC                 16
LEVEL3_CACHE_LINESIZE              64
LEVEL4_CACHE_SIZE                  0
LEVEL4_CACHE_ASSOC                 0
LEVEL4_CACHE_LINESIZE              0
```

- Tools : `lscpu`, `dmidecode`, `lshw`, `getconf`
- Different tools may aggregate information differently (e.g., how total as opposed to per-core cache is reported)
- L1d cache is for data, L1i is for instructions (instructions must live somewhere in the cache too!)

CPU cache

```
.... .... .... .... .... .... .... ....  .... .... .... .... .... SSSS SS11 1111
```

- For a 64 byte cache line, the least 6 bits of the address refer to the location inside the cache line. Not relevant in determining parking spot in the cache
- If we have 32kb of L1d cache, with a 64 byte line, we have 512 "parking spots" (lines)
- An 8 way associative cache will then have $512/8 = 64$ sets, and use the further 6 bits of a memory address to assign a set
- If we keep accessing random places in memory, it is very easy to run out of L1 cache: in the example here, we have only 64 sets!
- Address bits higher than the least 12 are not used in determining where in the cache a value is stored: any two addresses separated by 2^{12} map to the same set in the L1 cache.
- Variables with memory addresses separated by $setcount \times linesize$ compete for the same cache line
- For better performance, one should strive to write code utilising the whole cache line before it is evicted

Memory access patterns

```cpp
std::vector<int> A(N * N, 0);
for (size_t i = 0; i < N; ++i) {
    for (size_t j = 0; j < N; ++j) {
        A[i * N + j] += j + i;
    }
}
```

```cpp
for (size_t i = 0; i < N; ++i) {
    for (size_t j = 0; j < N; ++j) {
        A[j * N + i] += j + i;
    }
}
```

```cpp
for (size_t i = 0; i < N * N; ++i) {
    A[ pos[i] ] += i;
}
```

See also: CppCon 2016: Timur Doumler "Want fast C++? Know your hardware!"

- Q: Which way of accessing the "matrix" is faster, and by how much ?
- A: For N=10000, my laptop takes about 0.037 seconds for the row major pattern (top), and about 0.26 seconds for the column major pattern (middle), and 1.86 seconds for random pattern (bottom)

Memory

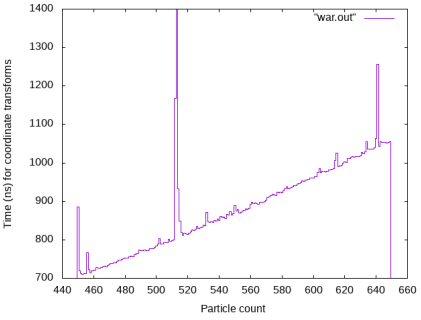
```
1  constexpr size_t size = 2 << 26;
2  std::vector< long > A(size, 0);
3  for (size_t step = 1; step <= 2048; step *= 2) {
4    for (size_t i = 0; i < size; i += step) A[i]++;
5  }
```

Step	Time
1	0.0967211
2	0.0943902
4	0.0929546
8	0.113927
16	0.137341
32	0.120449
64	0.0675447
128	0.0415029
256	0.016718
512	0.00694461
1024	0.00357155
2048	0.00178591

- For small step sizes, increasing the number of writes to the array does not change the total time.
- Multiple accesses inside a cache line has minimal extra cost.

4K aliasing

```
1   // Layout :
2   // x0, x1, x2 ... xn-1, y0, y1 ... yn-1,
3   // z0, z1 ... zn-1, wx0, wx1 ... wxn-1,
4   // wy0, wy1 ... wyn-1, wz0 ... wzn-1
5
6   for (size_t i=0;i<npart;++i) {
7     wx(i)=R(0,0)*x(i)+R(0,1)*y(i)+R(0,2)*z(i));
8     wy(i)=R(1,0)*x(i)+R(1,1)*y(i)+R(1,2)*z(i));
9     wz(i)=R(2,0)*x(i)+R(2,1)*y(i)+R(2,2)*z(i));
10  }
```



- Innocent looking code can sometimes produce weird changes in performance based on array sizes
- The spike in required time here comes for a particle count of about 512, when the different components of the data for one particle are separated by exactly 4kB.

Exercise 3.6: Memory effects

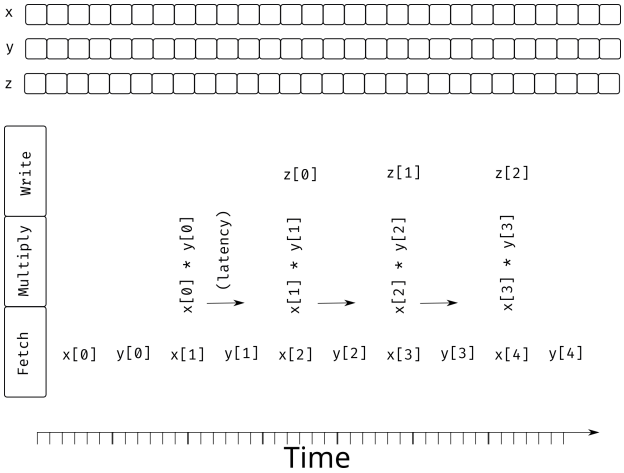
The following examples illustrate the cache effects discussed so far:

- `traverse0.cc` can be used to compare contiguous and non-contiguous access of a large array
- `every_nth.cc` compares times for accessing every n'th element, and highlights the cache line
- `transpose.cc` transpose operation on a matrix, which involves lots of non-contiguous access

Recommendations

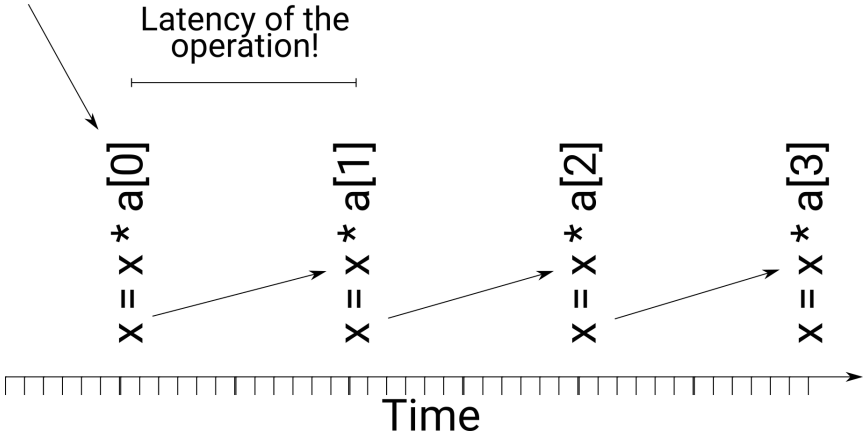
- Prefer `std::array` and `std::vector` for all your container needs as a default. Many libraries also provide other containers with contiguous storage providing advantages for specific use cases. Anything with non-contiguous storage needs to be carefully justified
- Organise code to maximise the use of any cache line that has been fetched:
  - Collate processing of nearby memory locations
  - Organise data structures so that things processed together are also stored near each other
- Keep variables as local as possible

Instruction pipeline

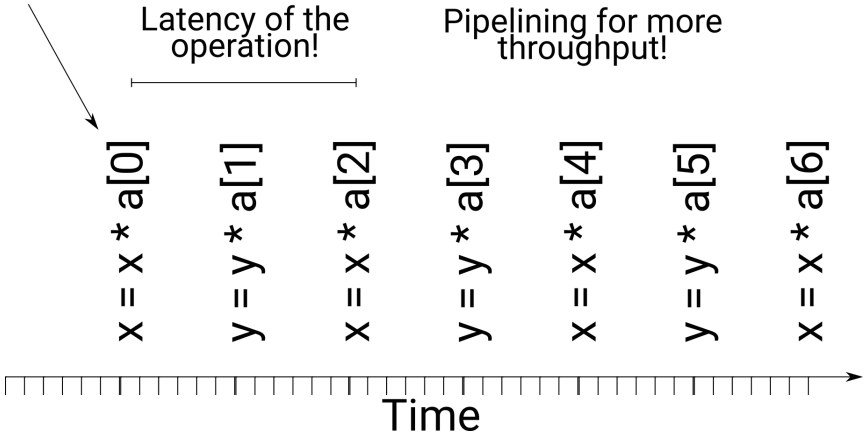


- A processor consists of many units, responsible for different actions, e.g., fetching instructions or data from memory, arithmetics, writing computed results back to memory
- When executing a program, pipelining helps keep different units busy throughout, improving throughput

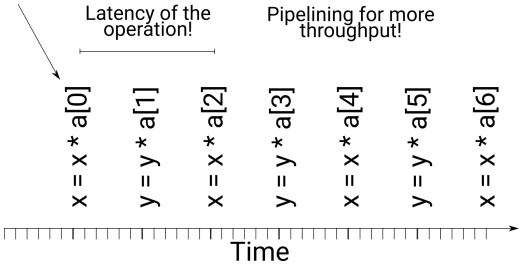
Instruction pipeline



Instruction pipeline



Instruction pipeline



- CPUs do not have to wait until one calculation is totally finished before starting another
- They "pipeline". If the data required for another instruction is available, that can start execution before the first is finished
- Because of pipelining, the processors are able to perform more operations in time Δt than $\frac{\Delta t}{t_{latency}}$
- Data dependencies create stalls in the pipeline
- Some modern processors even execute instructions "out of order", to keep the pipeline busy

Exercise 3.7:

The program `examples/ilp.cc` demonstrates the effects of data dependencies. Two alternative versions of a loop are given, performing the same total number of computations. One of them runs more than 5 times faster, because it avoids dependencies between successive calculations.

Pipeline and branched code

- Program execution flows through different units responsible for different work
- Branching can introduce holes in the pipeline without some workaround

- Instruction fetch
- Instruction decode
- Instruction execute
- Memory access
- Register write back

```
1  if (x+y>5) f();
2  else g();
```

- request mem x
- request mem y
- calc x+y
- calc res $>$ 5
- ?

The "next instruction" depends on the outcome of an instruction.

JÜLICH Forschungszentrum

Branch prediction

```
1   for (int i = 0; i < N; ++i) {
2       if (p[i] > gen()) {
3           b[i] = a[i] + c[i];
4           ++fwd;
5       } else {
6           a[i] = b[i] + c[i];
7           ++rev;
8       }
9   }
10  nngb = 0;
11  while (a) {
12      dist[nngb++] = distf(a,i);
13  }
```

- For efficient execution, different units in the pipeline must be kept busy as much as possible

- When branches are encountered, the CPU simply guesses which way it will go, and fetches instructions accordingly
- If the guess is right, no pipeline stall
- If it is wrong, all operations done with that guess must be purged

Branch mis-prediction penalty

```
1   for (int i = 0; i < N; ++i) {
2       if (p[i] > gen()) {
3           a[i] = (b[i] > r0 && b[i] < r1
4                               && c[i] < b[i]);
5       } else {
6           a[i] = b[i] + c[i];
7           ++rev;
8       }
9   }
10  nngb=0;
11  while (a) {
12      dist[nngb++] = distf(a,i);
13  }
```

- If statements, switches, loops contain obvious branches
- The ternary operator `a = cond ? v1 : v2` is (sometimes) a branch

- Not so obvious branches include boolean `||` and `&&` operators:
  - In a sequence of operations like `a || b || c || ...`, the operands are evaluated left to right until the first true value is obtained
  - In a sequence of operations like `a && b && c && ...`, the operands are evaluated left to right until the first false value is obtained

Not branches

```
1  auto f(int i) -> int
2  {
3      static const int a[4]={4,3,2,1};
4      int ans=0;
5      ans += (a[1]<i)?1:2;
6      return ans;
7  }
```

```
1  0000000000000000 <_Z1fi>:
2    cmp    edi,0x4
3    setl   al
4    movzx  eax,al
5    inc    eax
6    ret
```

- Conditional assignments are often reorganised as simple sequential instructions by compilers using special assembler instructions when available
- Loops with small loop counts may be automatically unrolled at compile time leaving simple linear code

```
1  0000000000000000 <_Z1fdPd>:
2    subsd  xmm0,QWORD PTR [rdi]
3    subsd  xmm0,QWORD PTR [rdi+0x8]
4    subsd  xmm0,QWORD PTR [rdi+0x10]
5    subsd  xmm0,QWORD PTR [rdi+0x18]
6    ret
```

Not branches

```
1  auto f(double x, double A[4]) -> double
2  {
3      double a=x;
4      for (int i=0;i<4;++i) a-=A[i];
5      return a;
6  }
```

- Conditional assignments are often reorganised as simple sequential instructions by compilers using special assembler instructions when available
- Loops with small loop counts may be automatically unrolled at compile time leaving simple linear code

```
1  0000000000000000 <_Z1fdPd>:
2    subsd  xmm0,QWORD PTR [rdi]
3    subsd  xmm0,QWORD PTR [rdi+0x8]
4    subsd  xmm0,QWORD PTR [rdi+0x10]
5    subsd  xmm0,QWORD PTR [rdi+0x18]
6    ret
```

Exercise 3.8:

Branch prediction effectiveness using the example program `examples/branch_prediction.cc`, compare the processor on your own computer with the processors on JURECA login nodes or compute nodes. The program partitions an array of integers into 3 ranges. Running it with a command line argument (value ignored) causes it to first sort the array and then perform the same partitioning actions. In the sorted array, the branches are easier to predict. What do you observe ? How do different compilers compare ?

Exercise 3.9:

The program `examples/branch_prediction1.cc` further illustrates hardware branch prediction. Here, two alternative kinds of calculations need to be done and accumulated separately. Depending on the value inside a random array of numbers, we decide between the two calculations. It is impossible for the compiler to pre-determine the branches. Adjust the threshold to shift the probability of the two branches and observe the performance. Again, compare the 3 compilers!

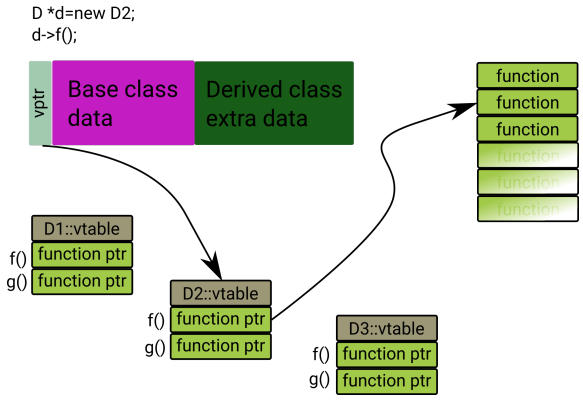
Class inheritance with virtual functions

```
1  auto main() -> int
2  {
3    vector<unique_ptr<Shape>> shapes;
4    shapes.push_back(make_unique<Circle>(0.5, Point(3,7)));
5    shapes.push_back(make_unique<Triangle>(Point(1,2),Point(3,3),Point(2.5,0)));
6  ...
7    for (auto&& shape : shapes) {
8      std::cout << shape->area() << '\n';
9    }
10 }
```

- A [smart] pointer to a base class is allowed to point to an object of a derived class
- Here, `shape[0]->area()` will call `Circle::area()`, `shape[1]->area()` will call `Triangle::area()`
- But, how does it work ?

JÜLICH Forschungszentrum

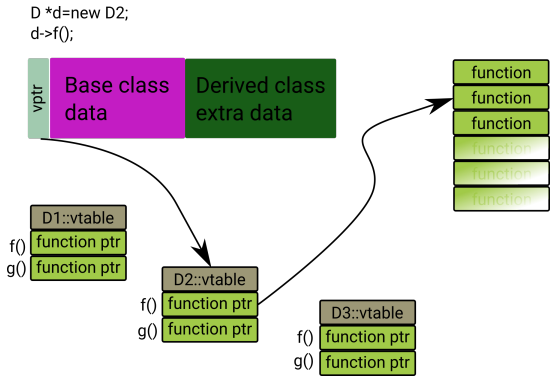
Calling virtual functions: how it works



- For classes with virtual functions, the compiler inserts an invisible pointer member to the data and additional book keeping code
- There is a table of virtual functions for each derived class, with entries pointing to function code somewhere
- The `vptr` pointer points to the *vtable* of that particular class

Calling virtual functions: how it works

- Virtual function call proceeds by first finding the right *vtable*, then the correct entry for the called function, dereferencing that function pointer and then executing the correct function body
- Branch mispredictions, cache misses ...
- For HPC applications, use of virtual functions in hot sections will hurt performance



- Often, the polymorphic behaviour sought after using virtual functions can be implemented with CRTP without the virtual function overhead

Polymorphism without virtual functions

- We have already seen how function overloading gives us a polymorphic unit : the overload set
- Different variation of a function is picked based on the type of the input parameters, or the constraints satisfied by the input parameters
- This is one kind of *static polymorphism*

Tag dispatching

```cpp
struct flying {};
struct swimming {};
template <class T>
void do_something(T && t, flying)
{
  t.fly(a,b);
}
template <class T>
void do_something(T && t, swimming) {...}
//...
template <class T>
void do_something(T t)
{
  do_something(t, typename T::preferred_mode{});
}
```

```cpp
class Buzzard {
public:
  using preferred_mode = typename flying;
};
class Whale {
public:
  using preferred_mode = typename swimming;
};
//...
Buzzard b;
do_something(b);
Whale w;
do_something(w);
```

- Logically similar operations on different types, where the operations depend on certain properties of the types

Tag dispatching

```cpp
struct flying {};
struct swimming {};
template <class T>
void do_something(T && t, flying)
{
  t.fly(a,b);
}
template <class T>
void do_something(T && t, swimming) {...}
//...
template <class T>
void do_something(T t)
{
  do_something(t, typename T::preferred_mode{});
}
```

```cpp
class Buzzard {
public:
  using preferred_mode = typename flying;
};
class Whale {
public:
  using preferred_mode = typename swimming;
};
//...
Buzzard b;
do_something(b);
Whale w;
do_something(w);
```

- Logically similar operations on different types, where the operations depend on certain properties of the types
- "Dispatch" functions to guide the compiler to a suitable implementation based on a "tag" in the incomming type

Tag dispatching

```cpp
struct flying {};
struct swimming {};
template <class T>
void do_something(T && t, flying)
{
  t.fly(a,b);
}
template <class T>
void do_something(T && t, swimming) {...}
//...
template <class T>
void do_something(T t)
{
  do_something(t, typename T::preferred_mode{});
}
```

```cpp
class Buzzard {
public:
  using preferred_mode = typename flying;
};
class Whale {
public:
  using preferred_mode = typename swimming;
};
//...
Buzzard b;
do_something(b);
Whale w;
do_something(w);
```

- Logically similar operations on different types, where the operations depend on certain properties of the types
- "Dispatch" functions to guide the compiler to a suitable implementation based on a "tag" in the incomming type
- Does not tie the overload to a specific type: dispatches based on some property of the input type

Choosing algorithm based on API

```
1  template <class C>
2  auto algo(C&& x) -> size_t {
3      if constexpr (hasAPI<C>) {
4          x.helper();
5          return x.calculateFast();
6      } else {
7          return x.calculate();
8      }
9  }
```

- We want to write a general algorithm for an operation
- In case the function argument has a certain member function, we have a neat and quick solution
- Otherwise, we have a fallback solution

Choosing algorithm based on API

```
1  template <class T>
2  concept FastCalculator = requires (T rex) {
3    { rex.calculateFast() };
4    { rex.helper() };
5  };
6  template <FastCalculator C> auto algo(C && x)
7  {
8      x.helper();
9      return x.calculateFast();
10 }
11 template <class C> auto algo(C && x)
12 {
13     return x.calculate();
14 }
```

- Write a `concept` describing what member functions, inner types (like `value_type` for iterators) an object should have to satisfy the API
- Overload based on whether the constraints are satisfied!

JÜLICH Forschungszentrum

Choosing algorithm based on API

```
1  auto main() -> int
2  {
3      Machinery obj;
4      auto res = algo(obj);
5      std::cout << "Result = " << res << "\n";
6  }
```

- Users of our great algorithm can simply call our `algo()` in their code
- If there is a `calculate` function, everything will work.
- If the author of the library providing `Machinery` goes on to implement `calculateFast` in the `Machinery` class, without any changes on the client side, or in the `algo` function, the compiler will make sure that the (hopefully) better, `calculateFast` function is used

Exercise 3.10:

The folder `examples/apishimming` contains the example `hasAPI` template function used in this section, with an application that uses it. By freeing the commented implementation of `calculateFast`, and recompiling, you will see that the call to `algo` automatically switches to use `calculateFast`.

```cpp
template <class T> struct hasAPI_t {
    using basetype = typename remove_reference<T>::type;
    template <class C> static constexpr auto test(C * x) ->
        decltype(x->calculateFast(),
        x->helper(),
        bool{})
    {
        return true;
    }
    static constexpr auto test(...) -> bool
    {
        return false;
    }
    static constexpr auto value =
        test(static_cast<basetype*>(nullptr));
};
template <class T>
constexpr bool hasAPI = hasAPI_t<T>::value;
template <class C>
auto algo(C && x)
    -> std::enable_if_t< hasAPI<C>, size_t>
{
    x.helper();
    return x.calculateFast();
}
template <class C> auto algo(C && x)
  -> std::enable_if_t< !hasAPI<C>, size_t>
{
    return x.calculate();
}
```

Will get the job done.

```cpp
template <class T>
concept FastCalculator = requires (T rex) {
  { rex.calculateFast() };
  { rex.helper() };
};
template <FastCalculator C> auto algo(C && x)
{
    x.helper();
    return x.calculateFast();
}
template <class C> auto algo(C && x)
{
    return x.calculate();
}
```

Will get the job done and keep you sane.

JÜLICH Forschungszentrum

Curiously Recurring Template Pattern

- You need types A and B which have some properties in common, which can be calculated using similar data
- There are a few polymorphic functions, but conceptually A and B are so different that you don't expect to store them in a single pointer container
- The penalty of using virtual functions seems to matter
- Option 1: implement as totally different classes, just copy and paste the common functions

Curiously Recurring Template Pattern

- You need types A and B which have some properties in common, which can be calculated using similar data
- There are a few polymorphic functions, but conceptually A and B are so different that you don't expect to store them in a single pointer container
- The penalty of using virtual functions seems to matter
- Option 1: implement as totally different classes, just copy and paste the common functions
- Option 2: try the CRTP

Curiously Recurring Template Pattern

```
template <class D> struct ViewInterface {
    auto der() const -> const D * {
        return static_cast<const D *>(this);
    }
    auto begin() const {
    // Wont compile if D does not inherit from this
      return der()->begin_impl();
    }
    auto version() const -> int {
    // Non-polymorphic "common" function
      return 42;
    }
};
struct Atoi : public ViewInterface<Atoi> {
    auto begin_impl() const { return bg; }
};
struct List : public ViewInterface<List> {
    auto begin_impl() const -> string {
        return &basenode;
    }
};
```

```
template <class T>
auto proc(ViewInterface<T> v) {
    auto b = v.begin();
    // ...
}
auto main() -> int {
  List H;
  proc(H);
  proc(Atoi{33});
}
```

- A function can demand that the inputs have a particular interface defined in the CRTP base
- Any input type inheriting from the CRTP base will be usable
- Polymorphism without virtual functions
- Enforces an interface at compile time
- Usually faster than virtual functions

“Mixin”

```cpp
// examples/crtp3.cc
template <class Derived> struct EnableCheckedAccess {
    auto at(std::size_t i) const {
        auto* d = static_cast<const Derived*>(this);
        if (i >= d->size())
            throw std::out_of_range(
                std::format("Index {} is out of range for container size {}", i, d->size()));
        return (*d)[i];
    }
};
struct MyVec : EnableCheckedAccess<MyVec> {
    auto operator[](std::size_t i) const { return i * i; }
    auto size() const -> std::size_t { return 5UL; }
};
auto main(int argc, char* argv[]) -> int {
    auto lim = argc > 1 ? std::stoul(argv[1]) : 5UL;
    MyVec v;
    try {
        for (auto i = 0UL; i < lim; ++i)
            std::print("Index = {}, value = {} \n", i, v.at(i));
    } catch (std::exception& err) { std::print("{}\n", err.what()); }
}
```

- Statically inject functionality into classes
- No virtual dispatch required

“Mixin”

```cpp
// examples/crtp4.cc
struct EnableCheckedAccess {
    template <class Self>
    auto at(this Self&& self, std::size_t i) {
        if (i >= self.size())
            throw std::out_of_range(
                std::format("Index {} is out of range for container size {}", i, self.size()));
        return self[i];
    }
};
struct MyVec : EnableCheckedAccess {
    auto operator[](std::size_t i) const { return i * i; }
    auto size() const -> std::size_t { return 5UL; }
};
auto main(int argc, char* argv[]) -> int {
    auto lim = argc > 1 ? std::stoul(argv[1]) : 5UL;
    MyVec v;
    try {
        for (auto i = 0UL; i < lim; ++i)
            std::print("Index = {}, value = {} \n", i, v.at(i));
    } catch (std::exception& err) { std::print("{}\n", err.what()); }
}
```

- Using the deducing this feature of C++23, we can make it much less weird!

Expression Templates

```
template <class T>
class vec {
  std::vector<T> dat;
public:
  vec(size_t n) : dat(n) {}
  auto operator[](size_t i) const -> T {
    return dat[i];
  }
  auto operator[](size_t i) -> T & {
    return dat[i];
  }
  size_t size() const{return dat.size();}
};
template <class T>
auto operator+(const vec<T> & v1,
               const vec<T> & v2) -> vec<T>
{
  assert(v1.size() == v2.size());
  auto ans = v1;
  for (size_t i = 0; i < ans.size(); ++i)
    ans[i] += v2[i];
  return ans;
}
```

```
vec<double> W(N), X(N), Y(N), Z(N);
//..
W = a * X + 2 * a * Y + 3 * a * Z;
```

- Naive implementation which expresses our intent elegantly
- Each multiplication and addition creates a temporary and does a loop over elements
- Poor performance

JÜLICH Forschungszentrum

Expression templates

If only we had a special class ...

- ... which stored references to `X`, `Y` and `Z`
- and had an `operator[]` which returns `a * X[i] + 2 * a * Y[i] + 3 * a * Z[i]`
- We could equip our `vec` class with a special assignment operator taking this special class as the right hand side

```
1  template <class T>
2  class vec {
3    template <class XPR>
4    auto operator=(const XPR & r) -> vec &
5    {
6      for (size_t i = 0; i < size(); ++i) {
7        dat[i] = r[i]; // and r[i] returns a*X[i]+2*a*Y[i]+3*a*Z[i]
8      } // One single loop, no temporaries
9      return *this;
10   }
11 };
```

- We need a different special class for every expression we have to evaluate

Expression templates

- If we make a class like :

```
template <class LHS, class RHS>
class vecsum {
    const LHS & lhs;
    const RHS & rhs;
public:
    vecsum(const LHS & l, const RHS & r) : lhs{l}, rhs{r} {
        assert(l.size() == r.size());
    }
    auto operator[](size_t i) const { return lhs[i] + rhs[i]; }
    auto size() const { return lhs.size(); }
};
```

- We can define the sum of two `vec` objects to be a `vecsum` type

```
template <class LHS, class RHS>
auto operator+(const LHS& v1, const RHS& v2) -> vecsum<LHS, RHS>
{
    return vecsum<LHS, RHS>(v1, v2);
}
```


JÜLICH Forschungszentrum

Expression templates

- If we try `vec1 + vec2`, no evaluation happens, and we get a `vecsum<vec, vec>` object, we can call `[]` on this object and cause the calculation to happen.
- But, if we try `vec1 + 54` or `34 + "dino"`, we get nonsensical compound objects
- If we write our `operator+` like :

```
template <class LHS, class RHS>
auto operator+(const expr<LHS> & v1, const expr<RHS> & v2) -> vecsum<LHS, RHS>
{
    return vecsum<LHS, RHS>(v1, v2);
}
```

  , we can restrict the template to objects which match the pattern `expr<something>`
- If we further want composability of the operations, we need `vecsum<LHS, RHS>` to also match the pattern `expr<something>`

Expression templates

Design with CRTP

- CRTP: a base template `vecxpr` to use as a base for all expressions of `vec` objects

```
template <class X> struct vecxpr {
    X& der() noexcept { return *static_cast<X*>(this); }
    const X& der() const { return *static_cast<const X*>(this); }
};
```


JÜLICH Forschungszentrum

Expression templates

Design with CRTP

- We make our expression classes like `vecsum` inherit from the template `vecxpr` instantiated on themselves:

```
1  template <class T1, class T2> class vecsum : public vecxpr<vecsum<T1,T2>> {
2    const T1 & lhs;
3    const T2 & rhs;
4  public:
5    using value_type = typename T1::value_type;
6    vecsum(const vecxpr<T1> & l, const vecxpr<T2> & r) : lhs{ l.der() }, rhs{ r.der() } {
7      assert(lhs.size() == rhs.size());
8    }
9    const auto operator[](size_t i) const { return lhs[i] + rhs[i]; }
10   size_t size() const { return lhs.size(); }
11 };
```

Expression templates

Design with CRTP

- `operator+` can now be written as:

```
1  template <class T1, class T2>
2  auto operator+(const vecxpr<T1> & v1, const vecxpr<T2> & v2)
3          -> const vecsum<T1, T2> {
4      return vecsum<T1, T2>{ l, r };
5  }
```


JÜLICH Forschungszentrum

Expression templates

Design with CRTP

- We also make the original `vec` class inherit from `vecxpr`

```
template <class T> class vec : public vecxpr<vec<T>> {
  std::vector<T> dat;
public:
    using value_type = T;
    vec(size_t n) : dat(n) {}
    auto operator[](size_t i) const -> const T& { return dat[i]; }
    auto operator[](size_t i) -> T& { return dat[i]; }
    size_t size() const { return dat.size(); }
    size_t n_ops() const { return 0; }
    template <class X>
    auto operator=(const vecxpr<X> & y) -> vec & {
        dat.resize(y.der().size());
        for (size_t i = 0; i < y.size(); ++i)
            dat[i] = y.der()[i];
        return *this;
    }
};
```

Expression templates

Design with CRTP

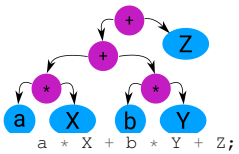
- We also make the original `vec` class inherit from `vecxpr`

```
1  template <class T> class vec : public vecxpr<vec<T>> {
2    std::vector<T> dat;
3  public:
4      using value_type = T;
5      vec(size_t n) : dat(n) {}
6      auto operator[](size_t i) const -> const T& { return dat[i]; }
7      auto operator[](size_t i) -> T& { return dat[i]; }
8      size_t size() const { return dat.size(); }
9      size_t n_ops() const { return 0; }
10     template <class X>
11     auto operator=(const vecxpr<X> & y) -> vec & {
12         dat.resize(y.der().size());
13         for (size_t i = 0; i < y.size(); ++i)
14             dat[i] = y.der()[i];
15         return *this;
16     }
17 };
```

- Notice the special assignment operator from an expression!

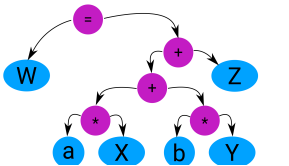
JÜLICH Forschungszentrum

Expression templates



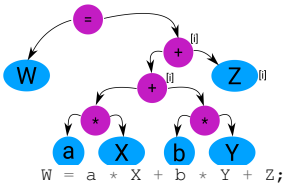

```
vecsum<
  vecsum<
    vecscl<vec<double>>,
    vecscl<vec<double>>
  >,
  vec<double>> ({{a,X},{b,Y}},Z);
// Let's call this type EXPR
```

Expression templates



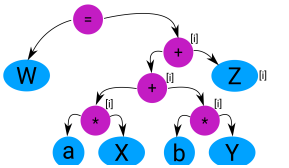

```
vec<double> &
vec<double>::operator=(const EXPR & E)
{
    dat.resize(E.size());
    for (size_t i = 0; i < E.size(); ++i)
        dat[i] = E[i];
    return *this;
}
```

Expression templates



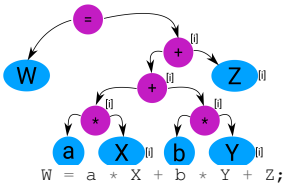

```
vec<double> &
vec<double>::operator=(const EXPR & E)
{
    dat.resize(E.size());
    for (size_t i = 0; i < E.size(); ++i)
        dat[i] = E[i];
const auto vecsum<L,R>::operator[](size_t i) const {
  return lhs[i] + rhs[i];
}
```

Expression templates




```
vec<double> &
vec<double>::operator=(const EXPR & E)
{
    dat.resize(E.size());
    for (size_t i = 0; i < E.size(); ++i)
        dat[i] = E[i];
const auto vecsum<L,R>::operator[](size_t i) const {
const auto vecscl<T>::operator[](size_t i) const {
    return lhs * rhs[i];
}
```

Expression templates




```
vec<double> &
vec<double>::operator=(const EXPR & E)
{
    dat.resize(E.size());
    for (size_t i = 0; i < E.size(); ++i)
        dat[i] = E[i];
const auto vecsum<L,R>::operator[](size_t i) const {
const auto vecscl<T>::operator[](size_t i) const {
const auto vec<T>::operator[](size_t i) const {
    return data[i];
}
```

Expression templates

- Elegant high level syntax
- Reduce temporaries
- Loop fusion
- Delayed evaluation: apply algorithmic optimizations on the entire expression, e.g.,
  - Evaluate `Matrix1 * Matrix2 * Vector` as `Matrix1 * (Matrix2 * Vector)`
  - Detect and eliminate cancelling operations, e.g., `Matrix_xpr1.transpose().transpose()`
  - Use optimized low level kernels with assembler, intrinsics, calls to vendor libraries etc to do the work
- However, can greatly increase compilation times

Exercise 3.11:

In `examples/xtmp0`, you will find a program which takes two numbers N and a as command line arguments, and creates 4 arrays W, X, Y, Z of size N (user defined array type `vec`). It fills X, Y and Z with random numbers and then calculates $W = a * X + 2 * a * Y + 3 * a * Z$, and times this operation by repeating the calculation 10 times. Two implementations of the user defined array type `vec` can be found: `naive_vec.hh` and `xtmp_vec0.hh`. Compile and run the program by alternating between the two headers. Study the code in `xtmp_vec0.hh`, which illustrates the ideas presented here about expression templates. The `xtmp_vec1.hh` implementation is almost the same, except using aligned allocation to store the arrays in the `vec` type. Test that as well.

Exercise 3.12:

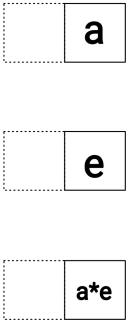
Introduce your own matrix class in the set up used in `examples/xtmp0`, so that matrix vector multiplications can be parts of vector expressions and `M1*M2*v` is evaluated as two matrix vector products rather than a matrix-matrix product followed by a matrix vector product.

Chapter 3

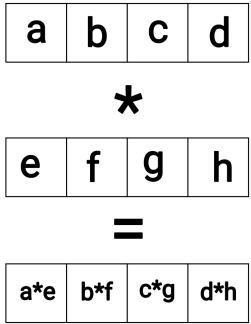
SIMD programming

JÜLICH Forschungszentrum

SIMD registers and operations



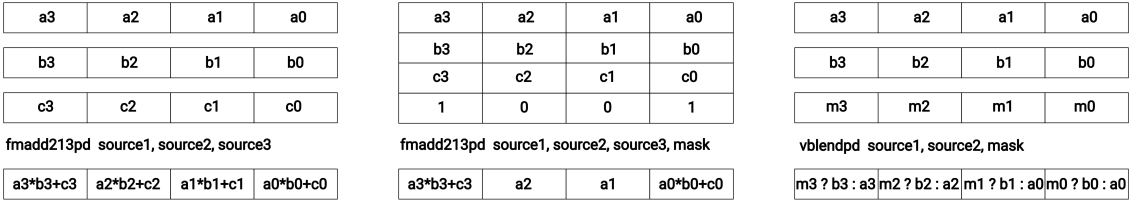
vmulsd xmm0, xmm0, xmm1



vmulpd ymm0, ymm0, ymm1

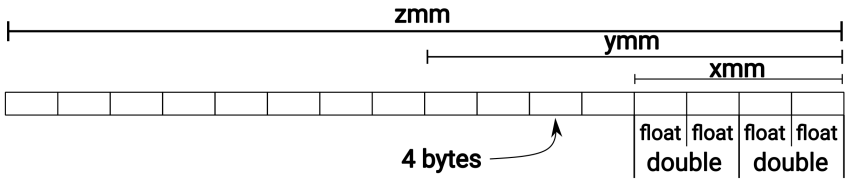
JÜLICH Forschungszentrum

SIMD registers and operations



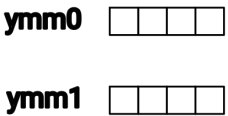
- Increasingly sophisticated instructions in newer CPUs
- Arithmatics, logical operations, shuffles, masked operations, trigonometry, cryptography ...

SIMD registers and operations



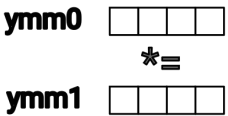
- `xmm0`, `xmm1`, ... `xmm7` (SSE)
- `xmm0` ... `xmm15`, `ymm0` ... `ymm15` (AVX, AVX2, FMA)
- `xmm0` ... `xmm31`, `ymm0` ... `ymm31`, `zmm0` ... `zmm31` (AVX512)

SIMD registers and operations



SIMD registers and operations

SIMD registers and operations



JÜLICH Forschungszentrum

SIMD registers and operations

Automatic vectorization

- Compilers try to automatically identify opportunities to use SIMD instructions and generate appropriate code
- We write code exactly (or at least more or less) as before, and the vectorizer brings more speed
- Sometimes you may have to be careful about alignment of the arrays (`alignas()`, `std::assume_aligned())`
- Sometimes you might need to indicate to the compiler that the multiple arrays involved in a loop do not overlap, can be assumed to be independent (`#pragma ivdep`)
- You may want to allow the compiler to proceed with the assumption that floating point arithmetic is associative (`-fassociative-math`)

Automatic vectorization

```
1  void f(double x[], double y[], unsigned N)
2  {
3      for (unsigned i=0U; i<N; ++i) x[i] = 5. * x[i] + y[i];
4  }
```

- Compiler asks : Can this loop be run in blocks of 4 or 8 for all inputs x and y ? What, if `y = x+1` !
- Then it makes careful decisions so that the results are correct for *every possible input*
- Sometimes, we don't care about every possible input. Our functions are often mere cogs in a bigger machine, and their contract is more limited

OpenMP SIMD directives

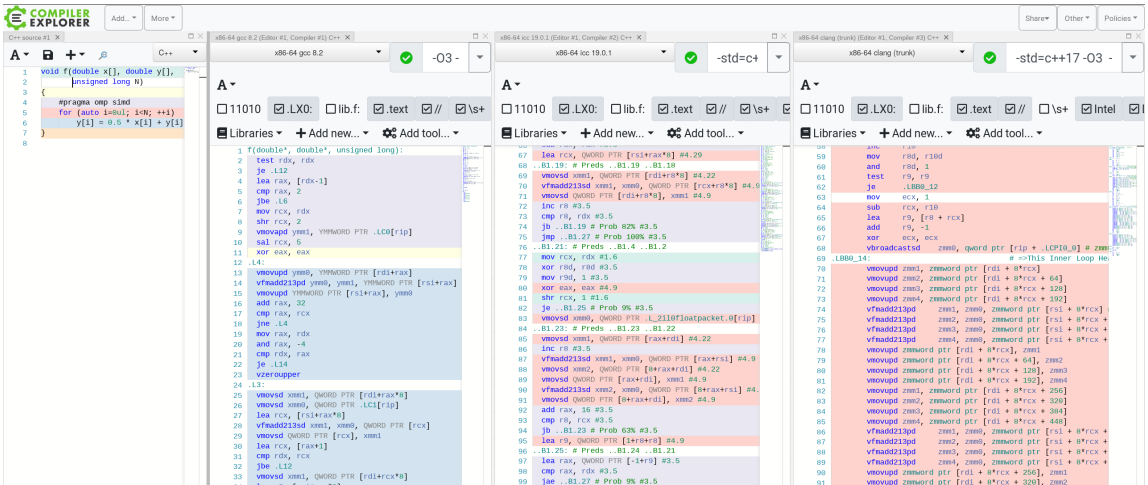
- Reorganize loop to run in chunks suitable for SIMD execution
- Syntax in C and C++ :

```
#pragma omp simd [clause [,clause ] ...]
for ( ... ) {}
```

- Often possible to call straight forward inline functions or vector enabled functions
  `#pragma omp declare simd`
- Can only be a traditional **for** loop. Loop count must be possible to determine at entry. No breaks.

```
template <typename T>
auto Vexv(T r2, T sigsa12) -> T {
    auto sg2
        = static_cast<T>(sqr(Lambda * sigsa12));
    auto a
        = static_cast<T>(icut2 * sqr(sigsa12));
    a = a * a * a;
    a = a * a;
    auto b = static_cast<T>(sixdivLLcut2 * a);
    a = 7.0 * a;
    T r6 = sg2 / r2;
    r6 = r6 * r6 * r6;
    return ksa * (r6 * r6 + a + b * r2);
}
auto addup( ___ ) -> double {
    double tot{};
    #pragma omp simd reduction(+:tot)
    for (size_t i=vec_size; i<R2.size(); ++i)
        tot += Vexv(R2[i], S12[i]);
    return tot;
}
```

For an excellent overview, search for "Michael Klemm, Intel, SIMD Vectorization with OpenMP"



JÜLICH Forschungszentrum

Digging deeper

```
1  double pairwise(unsigned i, unsigned j,
2                  SOA * particle_record)
3  {
4      // very clever calculations
5  }
6  auto energy() -> double
7  {
8    double ans = 0.;
9    for (auto i = 0; i < npt; ++i) {
10     #pragma omp please vectorize
11     for (auto j = i + 1; j < npt; ++j) {
12       ans +=
13         pairwise(i, j, my_particle_record);
14     }
15   }
16   return ans;
17 }
```

Convenient. But what are we not doing ?

- Coding to load groups of 2 or 4 or 8 numbers, working with them and storing the results
- Comparing different ways to use SIMD instructions to solve the problem for our actual inputs
- Choosing to use relaxed assumptions about floating point arithmetic at specific places in the code

Deviate only for special situations!

As HPC C++ programmers, we should know how to take full control of vectorization. But automatic or OpenMP based vectorization should be your first choice for production code. Most often they provide a cleaner, easier path. Sometimes, when the easier way does not provide enough low level access, we have ways to go beyond them.

R 1 3 5
2 4 6
12V

Wrapping intrinsics in zero (/low) cost abstractions

```
1  #include <immintrin.h>
2  union alignas(32) QuadDouble {
3      __m256d mm;
4      double d[4];
5      QuadDouble(__m256d oth) : mm{oth} {}
6      constexpr QuadDouble(double x, double y,double z=0., double t=0.) : d{x, y, z, t} {}
7
8      void aligned_load(double * v) {
9          assert(get_alignment(v) >= 32);
10         mm = _mm256_load_pd(v);
11     }
12     void unaligned_load(double * v) { mm = _mm256_loadu_pd(v); }
13
14     [[nodiscard]]auto operator[](unsigned i) const ->  double { return d[i%4]; }
15     auto operator[](unsigned i) -> double & { return d[i%4]; }
16
17     void operator=(double x) { mm = _mm256_broadcast_sd(&x); }
18     [[nodiscard]] auto horizontal_add() const -> double { return d[0] + d[1] + d[2] + d[3]; }
19 };
```


JÜLICH Forschungszentrum

Wrapping intrinsics in zero (/low) cost abstractions

```
1  auto get_alignment(void * var) {
2      auto n = reinterpret_cast<unsigned long>(var);
3      return (-n) & n;
4  }
5  auto operator+(QuadDouble a, QuadDouble b) -> QuadDouble {return _mm256_add_pd(a.mm, b.mm);}
6  auto operator-(QuadDouble a, QuadDouble b) -> QuadDouble {return _mm256_sub_pd(a.mm, b.mm);}
7  auto operator*(QuadDouble a, QuadDouble b) -> QuadDouble {return _mm256_mul_pd(a.mm, b.mm);}
8  auto operator/(QuadDouble a, QuadDouble b) -> QuadDouble {return _mm256_div_pd(a.mm, b.mm);}
9
10 auto main() -> int
11 {
12     QuadDouble a{3.1}, b{0.2, 5.4, 2.1, 9.8};
13     auto c = a * b - (a / b);
14     return c[2] < -1.;
15 }
```


JÜLICH Forschungszentrum

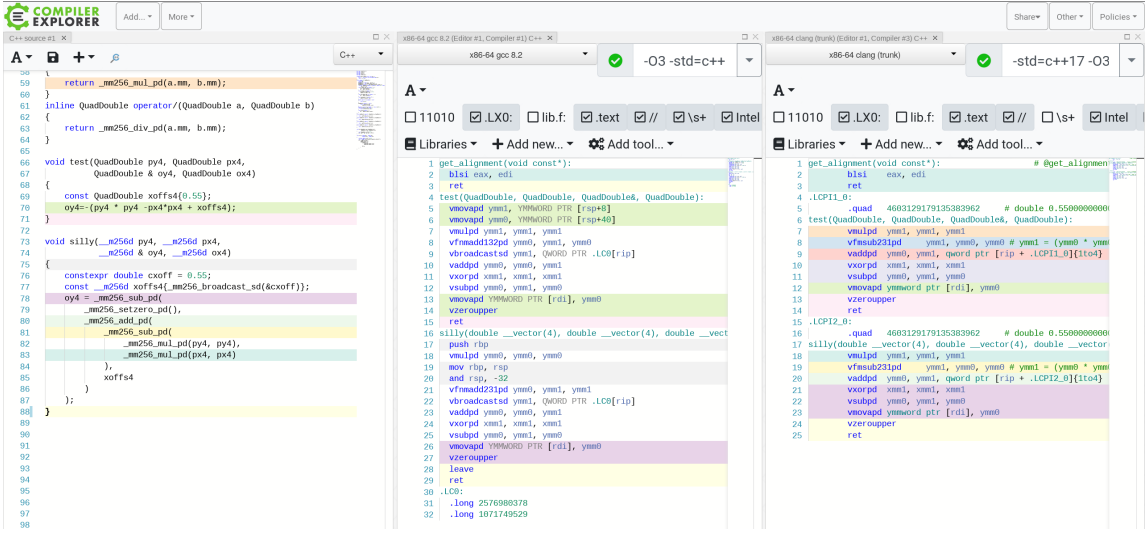
Wrapping intrinsics in zero(/low) cost abstractions

- Notational simplification, more readable and maintainable code, at no (or rather low) run time cost
- Need to wrap all operations used by your application (but only those)
- Need to hide vendor specific differences

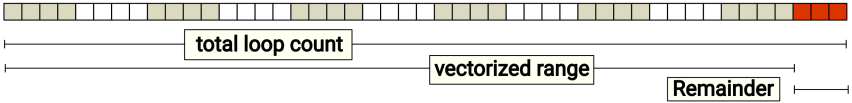
```
auto operator*(QuadDouble a, QuadDouble b) -> QuadDouble
{
    return _mm256_mul_pd(a.mm, b.mm);
}
```

```
1  # With Clang 7
2    operator*(QuadDouble, QuadDouble): # @operator*(QuadDouble, QuadDouble)
3      vmulpd  ymm0, ymm0, ymm1
4      ret
```

Note on alignment: Dynamically allocated arrays of our abstraction can cause unexpected crashes for C++98 . . . C++14, as the `new` operator could not align "over aligned" types on the heap. This was fixed in C++17, and optionally provided for C++11 and C++14 with compiler flags (GCC: `-faligned-new` Clang: `-faligned-allocation`).

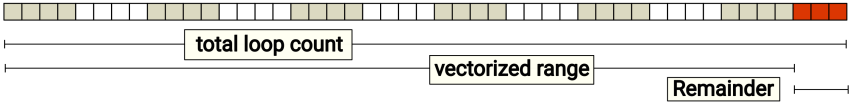

COMPILER EXPLORER
Add...
More
Share
Other
Policies
C++ source #1
x86-64 gcc 8.2 (Editor #1, Compiler #1) C++
x86-64 clang (trunk) (Editor #1, Compiler #3) C++
x86-64 gcc 8.2
-O3 -std=c++
x86-64 clang (trunk)
-std=c++17 -O3
Libraries
Add new...
Add tool...
11010
.LX0:
lib.f:
.text
//
\s+
Intel
return _mm256_mul_pd(a.mm, b.mm);
}
inline QuadDouble operator/(QuadDouble a, QuadDouble b)
{
return _mm256_div_pd(a.mm, b.mm);
}
void test(QuadDouble py4, QuadDouble px4,
QuadDouble & oy4, QuadDouble ox4)
{
const QuadDouble xoffs4(0.55);
oy4=-(py4 * py4 -px4*px4 + xoffs4);
}
void silly(__m256d py4, __m256d px4,
__m256d & oy4, __m256d ox4)
{
constexpr double cxoff = 0.55;
const __m256d xoffs4{_mm256_broadcast_sd(&cxoff)};
oy4 = _mm256_sub_pd(
_mm256_setzero_pd(),
_mm256_add_pd(
_mm256_sub_pd(
_mm256_mul_pd(py4, py4),
_mm256_mul_pd(px4, px4)
),
xoffs4
)
);
}
get_alignment(void const*):
blsi eax, edi
ret
test(QuadDouble, QuadDouble, QuadDouble&, QuadDouble):
vmovapd ymm1, YMMWORD PTR [rsp+8]
vmovapd ymm0, YMMWORD PTR [rsp+40]
vmulpd ymm1, ymm1, ymm1
vfnmadd132pd ymm0, ymm1, ymm0
vbroadcastsd ymm1, QWORD PTR .LC0[rip]
vaddpd ymm0, ymm0, ymm1
vxorpd xmm1, xmm1, xmm1
vsubpd ymm0, ymm1, ymm0
vmovapd YMMWORD PTR [rdi], ymm0
vzeroupper
ret
silly(double __vector(4), double __vector(4), double __vect
push rbp
vmulpd ymm0, ymm0, ymm0
mov rbp, rsp
and rsp, -32
vfnmadd231pd ymm0, ymm1, ymm1
vbroadcastsd ymm1, QWORD PTR .LC0[rip]
vaddpd ymm0, ymm0, ymm1
vxorpd xmm1, xmm1, xmm1
vsubpd ymm0, ymm1, ymm0
vmovapd YMMWORD PTR [rdi], ymm0
vzeroupper
leave
ret
.LC0:
.long 2576980378
.long 1071749529
get_alignment(void const*): # @get_alignment
blsi eax, edi
ret
.LCPI1_0:
.quad 4603129179135383962 # double 0.5500000000
test(QuadDouble, QuadDouble, QuadDouble&, QuadDouble):
vmulpd ymm1, ymm1, ymm1
vfmsub231pd ymm1, ymm0, ymm0 # ymm1 = (ymm0 * ymm
vaddpd ymm0, ymm1, qword ptr [rip + .LCPI1_0]{1to4}
vxorpd xmm1, xmm1, xmm1
vsubpd ymm0, ymm1, ymm0
vmovapd ymmword ptr [rdi], ymm0
vzeroupper
ret
.LCPI2_0:
.quad 4603129179135383962 # double 0.5500000000
silly(double __vector(4), double __vector(4), double __vector
vmulpd ymm1, ymm1, ymm1
vfmsub231pd ymm1, ymm0, ymm0 # ymm1 = (ymm0 * ymm
vaddpd ymm0, ymm1, qword ptr [rip + .LCPI2_0]{1to4}
vxorpd xmm1, xmm1, xmm1
vsubpd ymm0, ymm1, ymm0
vmovapd ymmword ptr [rdi], ymm0
vzeroupper
ret

Using our DIY SIMD library




```
1   // examples/diy/daxpy.cc
2   void daxpy_explicit(const std::vector<double> & x, std::vector<double> & y, double a) {
3       QuadDouble bx{0.}, by{0.};
4       const QuadDouble ba{a};
5       unsigned long vsize = x.size() - x.size() % 4;
6       const double * xptr0 = x.data();
7       const double * xptr1 = x.data() + vsize;
8       double * yptr = y.data();
9       for (; xptr0 != xptr1; xptr0 += 4, yptr += 4) {
10          bx.unaligned_load(xptr0);
11          by.unaligned_load(yptr);
12          by = by + bx * ba;
13          by.unaligned_store(yptr);
14      }
15      for (auto i=vsize; i<x.size(); ++i) y[i] += a* x[i];
16  }
```

Using our DIY SIMD library




```
1  // examples/diy/sprod.cc
2  #include "QuadDouble.hh"
3  auto sprod_explicit(size_t n, const double x[], const double y[]) -> double {
4      QuadDouble bx{0.}, by{0.}, tot{0.};
5      unsigned long vsize = n - n % 4;
6      const double * xptr0 = x;
7      const double * xptr1 = x + vsize;
8      for (; xptr0 != xptr1; xptr0 += 4, y += 4) {
9          bx.unaligned_load(xptr0);
10         by.unaligned_load(y);
11         tot = tot + bx * by;
12     }
13     auto res = tot.horizontal_add();
14     for (auto i = vsize; i < x.size(); ++i) res += x[i] * y[i];
15     return res;
16 }
```

Great! Now, what about AVX512 ? Power ? ARM ?

- Application code can operate using the abstraction
- Architecture specific details can be hidden inside the SIMD library
- No run-time indirection is needed. The compiler can be made to choose one specific version (macros, template specializations . . .)
- The author(s) of the SIMD library have to deal with the available capabilities in different instruction sets
- The library can also provide additional benefits: SIMD implementation of widely used functions, e.g., trigonometric, exponential functions

XSIMD

- C++ wrappers for SIMD intrinsics from "QuantStack". Include only. BSD-3-Clause license.
  `git clone https://github.com/QuantStack/xsimd.git`
- Abstractions for batches of values for SIMD calculations, e.g.,
  `xsimd::batch<double, xsimd::avx2>`

```
using Arch = xsimd::avx2;
xsimd::batch<double, Arch> x{1.,2.,3.,4.}, y{4.,3.,2.,1.};
std::cout << x + y << "\n";
```

- The second template argument can be left out. The default value: target architecture
- Vectorized forms of commonly used mathematical functions, such as trigonometric, exponential functions, error functions, e.g., `xsimd::asin(xsimd::batch<double, Arch>)`, `xsimd::exp(xsimd::batch<double, Arch>)`
- Regular arithmatic operations along with fma functions, e.g., `xsimd::fma(a, x, y)`
- Auto-detection and parametrisation based on available instruction set, e.g., based on vector width, `xsimd::batch<double, xsimd::avx2>`
- Aligned allocator:

```
template <class T> using myvector = std::vector<T, xsimd::aligned_allocator<T>>;
myvector<double> V(1000000, 1.2); // Aligned to cache line
```

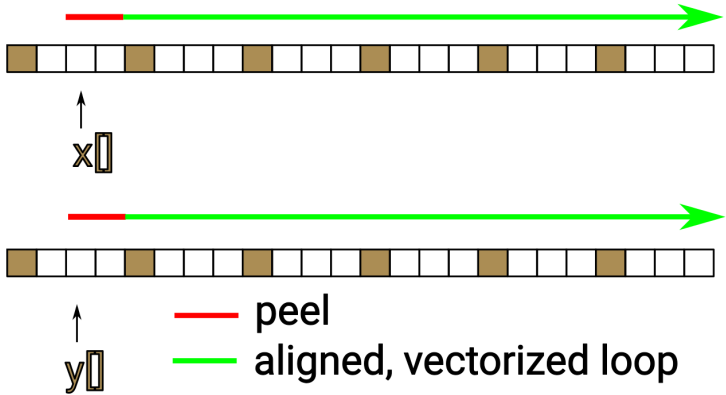
XSIMD

- Useful to write with a placeholder tag type `Arch`, to allow runtime architecture selection
- We will be using an alias `using btype = xsimd::batch<double, Arch>` in the following. Depending on the architecture, it may represent a batch of 2, 4 or 8 `double` values
- To load from an address in memory `xptr`, use `auto xb = btype::load_unaligned(xptr)`. If you know that the address is properly aligned for the batch, you can use `auto xb = btype::load_aligned(xptr)`. You can not load from an unaligned address using `load_aligned`.
- Loading can be controlled using a tag type: `auto xb = btype::load(xptr, alignment_tag)`, where `alignment_tag` is an object of one of the tag types `xsimd::aligned_mode` or `xsimd::unaligned_mode`.
- You can broadcast a scalar value to all positions in a SIMD batch like this: `auto ab = btype::broadcast(a);`.
- Batch objects can be combined using arithmetic operators, used in XSIMD mathematical functions etc to produce other batch objects
- To store the result to a location in memory, use the appropriate member function: `xb.store_unaligned(xptr)`, `xb.store_aligned(xptr)` or `xb.store(xptr, alignment_tag)`.

XSIMD

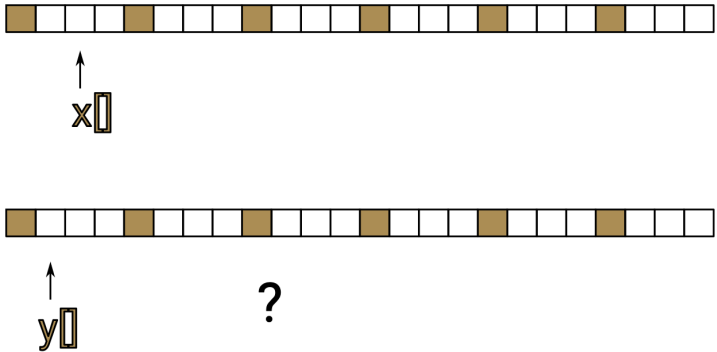
```
1  void daxpy(double a, std::span<const double> x, std::span<const double> y,
2             std::span<double> res) {
3      for (size_t i = 0UL; i < x.size(); ++i) {
4         res[i] = a * x[i] + y[i];
5      }
6  }
7  void daxpy_xsimd(double a, std::span<const double> x, std::span<const double> y,
8                   std::span<double> res) {
9      using btype = batch<double>;
10     constexpr auto vwidth = btype::size;
11     const auto ablk = btype::broadcast(a);
12     const auto vreps = x.size() - x.size() % vwidth;
13     for (size_t i = 0UL; i < vreps; i += vwidth) {
14         auto xblk = btype::load_unaligned(&x[i]);
15         auto yblk = btype::load_unaligned(&y[i]);
16         auto zblk = a * xblk + yblk;
17         zblk.store_unaligned(&res[i]);
18     }
19     for (size_t i = vreps; i < x.size(); ++i) { res[i] = a * x[i] + y[i]; }
20 }
```

Alignment and SIMD operations



Peel a few from the front and start aligned loads...

Alignment and SIMD operations



How many elements would you peel now ?

Alignment and SIMD operations

SIMD with complex numbers

- `std::complex<T>` has a fixed data layout, (*real*, *imag*) to be compatible with C
- Arrays of complex numbers have the real parts at non-adjacent, but statically predictable, locations (same applies to the imaginary parts)
- Many ways to code vectorized operations on complex numbers
- XSIMD (`batch<complex<double>, Arch>`) has abstractions for working with complex numbers
- Without such abstractions to aid us, explicit SIMD programming with complex number would be needlessly complicated

```
1  #include <xsimd/xsimd.hpp>
2  #include <complex>
3  #include <vector>
4  using namespace std;
5  void caxpy_xsimd(complex<double> a,
6          span<complex<double>> x,
7          span<const complex<double>> y)
8  {
9      using b_type =
10         xsimd::batch<complex<double>>;
11     b_type c = b_type::broadcast(a);
12     b_type xl, yl;
13     for (size_t i=0; i<x.size();
14                          i+=b_type::size) {
15         xl.load_unaligned(&x[i]);
16         yl.load_unaligned(&y[i]);
17         xl = c * xl + yl;
18         xl.store_unaligned(&x[i]);
19     }
20 }
```

XSIMD: architectures and dispatching

- It is possible to write programs for multiple architectures
- An appropriate instruction set is chosen based on architectures available at runtime
- Architecture adapted ("dispatched") functions are generated using `xsimd::dispatch()`
- Recipe:
  - Implement the function for a task as a functional with a template call operator
  - The template parameter `Arch` for the call operator serves the same purpose as our placeholder in the examples so far.
  - Generate a dispatched function using `xsimd::dispatch(functional)`
  - Use the return value of the dispatch function as a callable object with a signature without the `Arch` parameter.

```
1  struct daxpy_xsimd_t {
2    template <class Arch>
3    void operator()(Arch,
4        std::span<const double> x,
5        std::span<double> y,
6        double a) const
7    {
8      using b_type = xsimd::batch<double, Arch>;
9      b_type bx{}, by{};
10     const b_type ba{b_type::broadcast(a)};
11     // and so on...
12   }
13 };
14 inline auto daxpy_xsimd
15     = xsimd::dispatch(daxpy_xsimd_t{});
16 void elsewhere()
17 {
18   std::vector a(100UL, 4.3);
19   std::vector b(100UL, 3.2);
20   daxpy_xsimd(a, b, 8.0);
21 }
```

Exercise 4.1:

In the folder `examples/SIMD`, you will find several versions of a few short functions.

- Many examples here are not full programs and do not have `main` function.
- The DIY version does not require any libraries to compile, although it does need `immintrin.h`, which should be found in your system
- You should compile with the best available instruction set on your system (`-march=native` for GCC and Clang) and with optimization for speed
- The examples with XSIMD are in the next exercise.

Exercise 4.2:

`examples/SIMD/xsimd`: XSIMD demos

- For the examples with XSIMD, you will need to pass `-I /path/to/xsimdroot/include` to the compiler. There is nothing to link.

Example compile command :

```
$ g++ -std=c++23 -O3 -march=native -I $XSIMD_INCLUDE_DIR exvol1.cc -o exvol1.g
```

The examples `x0.cc`, `x1.cc` and `x2.cc` show the basic syntax. `x0.cc` shows an explicitly set architecture. But `x1.cc` and `x2.cc` use the default batch. Build them with and without `-march=native` and run them, to understand the role of the compiler option. The examples `exvol1.cc` and `daxpy1.cc` demonstrate architecture dispatching. The remaining two examples, `brighten.cc` and `nn_relu.cc` show two tiny applications: `brighten.cc` brightens the pixels of an image. The `nn_relu.cc` shows a single layer feed forward neural network with a reLU activation function.

C++26 SIMD types and operations

```cpp
#include <simd>
namespace simd = std::simd;
// Batch type like in XSIMD
simd::basic_vec<T, ABI>
// Convenient alias
simd::vec<T, N = automatic>

simd::vec<int> a = 1;
constexpr auto bsize = simd::vec<int>::size();
simd::vec<int> b([](int i) { return i - 2; });
auto c = a + b;
auto d = simd::select(c < 0 ? -c : c);
auto e = d * d;
auto inner_product = simd::reduce(e);
std::println("e", e);
std::vector<int> vi(100, 1);
a = simd::unchecked_load(vi.begin(), bsize);
simd::unchecked_store(a, vi.begin(), bsize);
```

- Most basic operations are already implemented in the incomplete implementation in GCC
- Can be written out using `std::print`
- Large parts of the proposal, including math functions for data parallel types are still not implemented.

Exercise 4.3:

The folder `examples/SIMD/std_simd` contains some of the programs corresponding to those in the XSIMD exercise, implemented using the C++ standard library SIMD functionality. This is accepted for inclusion in C++26, which is currently in a feature-freeze. But, there is a partial implementation in GCC. The README file in the directory contains lots of comments about the programs. At the moment, this is just a possible picture of what working with `std::simd` might look like.

Summary

- Directly coding with SIMD types exposes algorithmic challenges concerning vectorization
- We are much more directly in control
- Quite often, correctly done OpenMP will bring you most of the benefits, but, knowing how to work with intrinsics gives you a fallback option when the simple approach fails. At the very least, when you try to vectorize yourself, you might see why OpenMP didn't do as you had hoped.
- If you work with C++, use its strengths: strive for zero-overhead abstractions instead of resigning to a life of verbose and error-prone misery
- Alternatively, use a SIMD library with a compatible license
  - They aleady exist, and others have already created the necessary abstractions
  - They support multiple instruction sets and CPU architectures
  - Often come with vectorized versions of common mathematical functions

Chapter 4

Parallelisation using PSTL and TBB

JÜLICH Forschungszentrum

Parallel computing

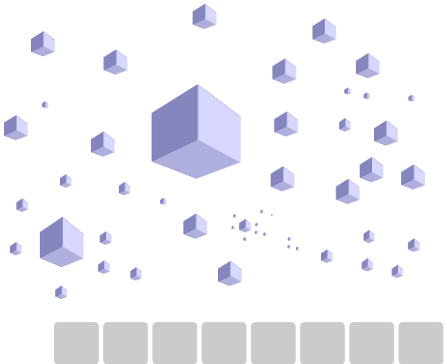
- Engineering (power consumption) challenges make processors with higher and higher clock rates impractical
- Computers in the last 20 years have instead increased processing power by adding more hardware for parallel processing

Parallel computing

```
1  auto gcd(unsigned s, unsigned l) -> unsigned
2  {
3      if (s > l)
4          std::swap(s, l);
5      while (s != 0) {
6          auto r = l % s;
7          l = s;
8          s = r;
9      }
10     return l;
11 }
```


- Engineering (power consumption) challenges make processors with higher and higher clock rates impractical
- Computers in the last 20 years have instead increased processing power by adding more hardware for parallel processing
- A sequence of dependent operations on a small set of entities is ill-suited for processing with many workers

Parallel computing



- Engineering (power consumption) challenges make processors with higher and higher clock rates impractical
- Computers in the last 20 years have instead increased processing power by adding more hardware for parallel processing
- A sequence of dependent operations on a small set of entities is ill-suited for processing with many workers
- Given a large amount of information to be processed, or a task with a large number of independent sub-tasks, it is possible to reduce the overall processing time.

Parallel computing

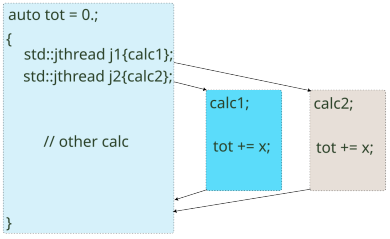
What mechanisms do we have in C++ to exploit available parallelism in hardware?

- Threads, mutexes, atomic operations
- RAII for resource management
- Libraries to partition and assign work to workers
- Templates, lambda functions, CTAD
- High-level STL style algorithms abstracting common programming building blocks
- Containers and allocators for more efficient (and corrrect) parallel processing

Threads

```
1  auto calc1 = [=]() {
2      auto tot1 = 0.;
3      for (auto i = 0UL; i < N; ++i) {
4          auto ang = 2 * i * pi / N;
5          tot1 += std::cos(ang) * std::cos(ang);
6      }
7  };
8  auto calc2 = [=]() {
9      auto tot1 = 0.;
10     for (auto i = 0UL; i < N; ++i) {
11         auto ang = 2 * i * pi / N;
12         tot1 += std::sin(ang) * std::sin(ang);
13     }
14 };
15 std::jthread j1 { calc1 };
16 std::jthread j2 { calc2 };
```

- `std::thread`, `std::async` ... since C++11
- Parallel algorithms since C++17
- `std::jthread`, `std::stop_token` since C++20
- `std::jthread` joins in the destructor

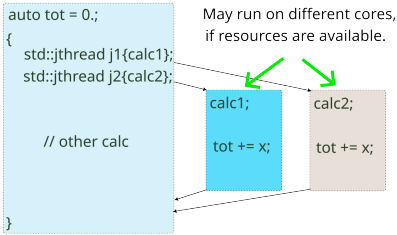


JÜLICH Forschungszentrum

Threads

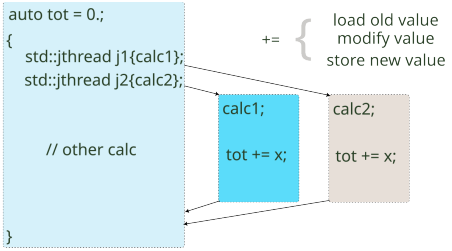
```
1  auto calc1 = [=]() {
2      auto tot1 = 0.;
3      for (auto i = 0UL; i < N; ++i) {
4          auto ang = 2 * i * pi / N;
5          tot1 += std::cos(ang) * std::cos(ang);
6      }
7  };
8  auto calc2 = [=]() {
9      auto tot1 = 0.;
10     for (auto i = 0UL; i < N; ++i) {
11         auto ang = 2 * i * pi / N;
12         tot1 += std::sin(ang) * std::sin(ang);
13     }
14 };
15 std::jthread j1 { calc1 };
16 std::jthread j2 { calc2 };
```

- `std::thread`, `std::async` ... since C++11
- Parallel algorithms since C++17
- `std::jthread`, `std::stop_token` since C++20
- `std::jthread` joins in the destructor



Threads

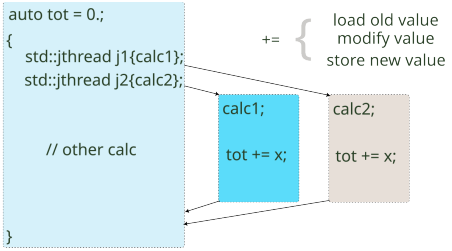
```
1  auto tot = 0.;
2  {
3      std::jthread j1 { [&]() {
4          for (auto i = 0UL; i < N; ++i) {
5              auto ang = 2 * i * pi / N;
6              tot += std::cos(ang) * std::cos(ang);
7          }
8      } };
9      std::jthread j2 { [&]() {
10         for (auto i = 0UL; i < N; ++i) {
11             auto ang = 2 * i * pi / N;
12             tot += std::sin(ang) * std::sin(ang);
13         }
14     } };
15 }
16 std::cout << "Total " << tot << "\n";
```



- Modification of data at the same address from multiple threads can lead to “data races”

Threads

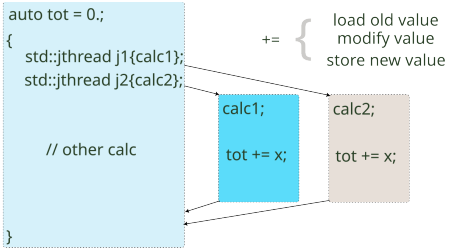
```
1  auto tot = 0.;
2  {
3      std::jthread j1 { [&]() {
4          for (auto i = 0UL; i < N; ++i) {
5              auto ang = 2 * i * pi / N;
6              tot += std::cos(ang) * std::cos(ang);
7          }
8      } };
9      std::jthread j2 { [&]() {
10         for (auto i = 0UL; i < N; ++i) {
11             auto ang = 2 * i * pi / N;
12             tot += std::sin(ang) * std::sin(ang);
13         }
14     } };
15 }
16 std::cout << "Total " << tot << "\n";
```



- The result can be incorrect, since the load-modify-commit operations from the two threads can overlap

Threads

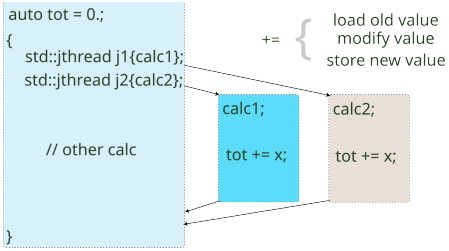
```
1   std::mutex totmutex;
2   {
3       std::jthread j1 { [&]() {
4           for (auto i = 0UL; i < N; ++i) {
5               auto ang = 2 * i * pi / N;
6               std::scoped_lock lck { totmutex };
7               tot += std::cos(ang) * std::cos(ang);
8           }
9       } };
10      std::jthread j2 { [&]() {
11          for (auto i = 0UL; i < N; ++i) {
12              auto ang = 2 * i * pi / N;
13              std::scoped_lock lck { totmutex };
14              tot += std::sin(ang) * std::sin(ang);
15          }
16      } };
17  }
18  std::cout << "Total " << tot << "\n";
```



- Fix 1: `std::mutex`: A resource which can be acquired by only one thread at a time. Must be released by the acquiring thread. `std::scoped_lock` manages mutex acquisition/release using RAII

Threads

```
1   std::atomic<double> tot {};
2   {
3       std::jthread j1 { [&]() {
4           for (auto i = 0UL; i < N; ++i) {
5               auto ang = 2 * i * pi / N;
6               tot += std::cos(ang) * std::cos(ang);
7           }
8       } };
9       std::jthread j2 { [&]() {
10          for (auto i = 0UL; i < N; ++i) {
11              auto ang = 2 * i * pi / N;
12              tot += std::sin(ang) * std::sin(ang);
13          }
14      } };
15  }
16  std::cout << "Total " << tot << "\n";
```



- `std::atomic<T>` gives us “atomic” load-modify-commit operations

Threads

```
1  struct wrapped1 {
2      int val {};
3  };
4  template <class W>
5  struct func {
6  void operator()(volatile W* var)
7  {
8      for (unsigned i = 0; i < WORKLOAD / PARALLEL; ++i) {
9          var->val = var->val + 1;
10     }
11 }
12 };
13 {
14    std::array<wrapped2, PARALLEL> arr {};
15    {
16        std::array<std::jthread, PARALLEL> threads;
17        for (unsigned i = 0U; i < PARALLEL; ++i) {
18            threads[i] =
19            std::jthread(func<wrapped2>{}, &arr[i]);
20        }
21    }
22 }
```

- Even when threads write to different addresses, there can be a significant slowdown because of “false sharing”

JÜLICH Forschungszentrum

Threads

```
1  struct align_as(std::hardware_destructive_interference_size)
2  wrapped1 {
3      int val {};
4  };
5  template <class W>
6  struct func {
7  void operator()(volatile W* var)
8  {
9      for (unsigned i = 0; i < WORKLOAD / PARALLEL; ++i) {
10         var->val = var->val + 1;
11     }
12 }
13 };
14 {
15    std::array<wrapped2, PARALLEL> arr {};
16    {
17        std::array<std::jthread, PARALLEL> threads;
18        for (unsigned i = 0U; i < PARALLEL; ++i) {
19            threads[i] =
20            std::jthread(func<wrapped2>{}, &arr[i]);
21        }
22    }
23 }
```

- Even when threads write to different addresses, there can be a significant slowdown because of “false sharing”
- Mitigation: alignment or padding

JÜLICH Forschungszentrum

Parallel STL

- Parallel versions of the high-level building blocks such as `std::sort`, `std::reduce` etc.
- C++17 parallel STL provides a way to express that something can be done in parallel, but does not mandate implementation strategy
- Programs already written using algorithms will offer many opportunities for exploiting parallelism
- A TBB based implementation is used since GCC 9.1. Intel and Microsoft compilers have their implementations as well.
- `std::sort` sorts.
  `std::sort(std::execution::par, ...)` sorts in parallel
- `std::reduce` adds up elements from a range.
  `std::reduce(std::execution::par, ...)` adds up elements in parallel

```
1  std::sort(std::execution::par,
2            points.begin(), points.end(),
3            [](auto p1, auto p2) {
4                return p1.x() < p2.x();
5            });
6  std::for_each(std::execution::par_unseq,
7                points.begin(), points.end(),
8                [](auto & p) {
9                    p.norm(1);
10               });
```

- As of GCC 15.2, to compile programs using parallel algorithms, we need to link with `libtbb` and `libtbbmalloc`, e.g.,
  `G par_user.cc -ltbb -ltbbmalloc`
- As of Clang 19.1, parallel STL remains an experimental feature in `libc++`, and must be enabled through `-fexperimental-library`

Execution policies

- `std::execution::sequenced_policy` : Parallel algorithm's execution may not be parallelised. Element wise operations are indeterminately sequenced in the calling thread. An instance called, `std::execution::seq` is usually used to disambiguate overload resolution
- `std::execution::parallel_policy` : May be parallelised. Element wise operations can happen in the calling thread, or on another. Relative sequencing is indeterminate. Convenience instance: `std::execution::par`
- `std::execution::parallel_unsequenced_policy` May be parallelised and vectorised. Element wise operations can run in unspecified threads, and can be unordered in each thread. `std::execution::par_unseq`
- `std::execution::unsequenced_policy` Only vectorised. `std::execution::unseq`

Parallel STL examples

Exercise 5.1:

The program `examples/pstl/inner_product.cc` demonstrates the use of the parallel STL library, performing a simple inner product calculation. Use `-ltbb -ltbbmalloc` for linking, or use the `CMake` file in the directory.

Exercise 5.2:

The program `examples/pstl/transform_reduce.cc` creates a vector of random points in 2D, and then calculates the moment of inertia using STL algorithms. Just switching the execution policy parameter, the ptogram can be parallelised and vectorised. Test!

Parallel STL examples

Exercise 5.3:

Parallelise the program `exercises/pstl/mandelbrot0.cc` using parallel STL.

Exercise 5.4:

At what size of a group of random strangers does the chance of two people sharing a birthday become greater than 0.5? The program `birthday_problem.cc` solves it using a crude, brute force Monte Carlo simulation. Parallelise it using parallel STL.

Examples in this section can be done with both GCC and Clang, with some caveats when using Clang.

```
clang++ -std=c++23 -stdlib=libc++ -fexpermental-library -O3 -march=native ___.cc
```

and

```
clang++ -std=c++23 -stdlib=libstdc++ -O3 -march=native ___.cc
```

will both will work. As of October 2025, `libc++` hasn't optimised performance when using parallel algorithms.

TBB: Threading Building Blocks I

- Provides utilities like `parallel_for`, `parallel_reduce` to simplify the most commonly used structures in parallel programs
- Provides scalable concurrent containers such as vectors, hash tables and queues for use in multi-threaded environments
- No direct support for vector parallelism. But can be combined with auto-parallelisation and `#pragma omp simd` etc or explicit SIMD with a SIMD library
- Supports complex models such as pipelines, data flow and unstructured task graphs
- Scalable memory allocation, avoidance of false sharing, thread local storage
- Low level synchronisation tools like mutexes and atomics
- Work stealing task scheduler
- `http://www.threadingbuildingblocks.org`
- Structured Parallel Programming, Michael McCool, Arch D. Robinson, James Reinders

Using TBB

- Public names are available under the namespaces `tbb` and `tbb::flow`
- You indicate "available parallelism", scheduler may run it in parallel if resources are available
- Unnecessary parallelism will be ignored

parallel invoke

```
1  void prep(Population &p);
2  void iomanage();
3  tbb::parallel_invoke(
4     [&] {
5         noise_w(0., pars.sigma, wns);
6         std::copy(wns.begin(), wns.end(), wnoisemat.begin());
7     },
8     [&] {
9         noise_phi(0., pars.sigma, phins);
10        std::copy(phins.begin(), phins.end(), phinoisemat.begin());
11    });
```

Exercise 5.5: examples/tbb/parallel_invoke.cc

Compile with

```
G parallel_invoke.cc -ltbb -ltbbmalloc
```

- A few adhoc tasks which do not depend on each other
- Runs them in parallel
- waits until all of them are finished

JÜLICH Forschungszentrum

TBB task groups

```
1  struct Equation {
2      void solve();
3  };
4
5  std::list<Equation> equations;
6  tbb::task_group g;
7  for (auto eq : equations)
8      g.run([]{eq.solve();});
9
10 g.wait();
```

- Run an arbitrary number of callable objects in parallel
- In case an exception is thrown, the task group is cancelled

TBB task arena

```
1  auto main(int argc, char *argv[]) -> int
2  {
3      size_t nthreads=std::stoul(argv[1]);
4      tbb::task_arena main_executor;
5      main_executor.initialize(nthreads);
6      main_executor.execute([&]{
7          haha();
8      });
9  }
10 void haha()
11 {
12     ...
13     tbb::parallel_invoke(a,b,c,d,e);
14 }
15 void a()
16 {
17     tbb::parallel_for(...);
18 }
```

- Task arena to manage tasks, maps them to threads etc.
- Number of threads in an arena limited by its concurrency level
- Execute function, with a function object as argument.
- Returns the same thing as the function it is executing.

Parallel for loops

- Template function modelled after the `for` loops, like many STL algorithms
- Takes a callable object as the third argument
- Using lambda functions, you can expose parallelism in sections of your code

```
1  tbb::parallel_for(first,last,f);
2  // parallel equivalent of
3  // for (auto i=first;i<last;++i) f(i);
4
5  tbb::parallel_for(first,last,stride,f);
6  // parallel equivalent of
7  // for (auto i=first;i<last;i+=stride)
8  //     f(i);
9
10 tbb::parallel_for(first,last,
11                   [captures](anything){
12     //Code that can run in parallel
13 });
14
```

Parallel for with ranges

- Splits `range` into smaller ranges, and applies `f` to them in parallel
- Possible to optimize `f` for sub-ranges rather than a single index
- Any type satisfying a few design conditions can be used as a range
- Multidimensional ranges possible

```
1  tbb::parallel_for(0,1000000,f);
2  // One parallel invocation for each i!
3  tbb::parallel_for(range,f);
4
5  // A type R can be a range if the
6  // following are available
7  R::R(const R &);
8  R::~R();
9  bool R::is_divisible() const;
10 bool R::empty() const;
11 R::R(R & r,split);//Split constructor
```


JÜLICH Forschungszentrum

Parallel for with ranges

```
1  tbb::blocked_range<int> r{0,30,20};
2  assert(r.is_divisible());
3  blocked_range<int> s{r};
4  //Splitting constructor
5  assert(!r.is_divisible());
6  assert(!s.is_divisible());
7
```

- `tbb::blocked_range<int>(0,4)` represents an integer range 0..4
- `tbb::blocked_range<int>(0,50,30)` represents two ranges, 0..25 and 26..50
  - So long as the size of the range is bigger than the "grain size" (third argument), the range is split

Parallel for with ranges

```
1  void daxpcy_tbb(double a, std::span<const double> x, std::span<double> y) {
2      tbb::parallel_for(tbb::blocked_range<int>(0, x.size()),
3                        [&](tbb::blocked_range<int> r) {
4          for (size_t i = r.begin(); i != r.end(); ++i) {
5              y[i] = a * sin(x[i]) + cos(y[i]);
6          }
7      });
8  }
```

- `parallel_for` with a range uses split constructor to split the range as far as possible, and then calls f(range), where `f` is the functional given to `parallel_for`
- It is unlikely that you wrote your useful functions with ranges compatible with `parallel_for` as arguments
- But with lambda functions, it is easy to fit the parts!

Exercise 5.6: TBB parallel for demo

The program `examples/dasxpcy.cc` demonstrates the use of parallel for in TBB. It is a slightly modified version of the commonly used DAXPY demos. Instead of calculating $y = a * x + y$ for scalar a and large vectors x and y, we calculate $y = a * sin(x) + cos(y)$. To compile, you need to load your compiler and TBB modules, and use them like this:

```
1  G dasxpcy.cc -ltbb -ltbbmalloc
```

2D ranges

```
1  void f(size_t i, size_t j);
2  tbb::blocked_range2d<size_t> r{0, N, 0, N};
3  tbb::parallel_for(r, [&](tbb::blocked_range2d<size_t> r) {
4      for (auto i = r.rows().begin(); i != r.rows().end(); ++i) {
5          for (auto j = r.cols().begin(); j != r.cols().end(); ++j) {
6              f(i, j);
7          }
8      }
9  });
```

- `rows()` is an object with a `begin()` and an `end()` returning just the integer row values in the range. Similarly: `cols()` ...
- 2D range can also be split
- The callable object argument should assume that the original 2D range has been split many times, and we are operating on a smaller range, whose properties can be accessed with these functions.

Parallel reductions with ranges

```
1  T result = tbb::parallel_reduce(range, identity, subrange_reduction, combine);
```

- `range` : As with parallel for
- `identity` : Identity element of type T. The type determines the type used to accumulate the result
- `subrange_reduction` : Functor taking a "subrange" and an initial value, returning reduction
- `combine` : Functor taking two arguments of type T and returning reduction over them over the subrange. Must be associative, but not necessarily commutative.

Parallel reduce with ranges

```
1  double inner_prod_tbb(std::span<const double> x, std::span<double> y) {
2      return tbb::parallel_reduce(
3            tbb::blocked_range<int>(0, n), // range
4            double{}, // identity
5            [&](tbb::blocked_range<int> &r, float in){
6                return std::inner_product(x.begin() + r.begin(), x.begin() + r.end(),
7                                          y.begin() + r.begin(), in);
8            }, // subrange reduction
9            std::plus<double>{} // combine
10     );
11 }
```

- With TBB ranges, we can use blocked implementations with hopefully vectorisable calculations in subranges
- Two functors are required, either of which could be lambda functions
- Important to add the contribution of initial value in subrange reductions

Exercise 5.7: TBB parallel reduce

The program `tbbreduce.cc` is a demo program to calculate an integral using `tbb::parallel_reduce`
What kind of speed up do you see relative to the serial version ? Does it make sense considering the number of physical cores in your computer ?

Enumerable thread specific

```
1  tbb::enumerable_thread_specific<double> E;
2  double Eglob=0;
3  double f(size_t i, size_t j);
4  tbb::blocked_range2d<size_t> r{0, N, 0, N};
5  tbb::parallel_for(r, [&](tbb::blocked_range2d<size_t> r){
6      auto & eloc = E.local();
7      for (size_t i = r.rows().begin(); i != r.rows().end(); ++i) {
8          for (size_t j = r.cols().begin();j != r.cols().end(); ++j) {
9              if (j > i) eloc += f(i,j);
10         }
11     }
12 });
13 Eglob = 0;
14 for (auto& v : E) {Eglob += v; v = 0;}
```

- Thread local "views" of a variable
- behaves like an STL container of those views
- Member function `local()` gives a reference to the local view in the current thread
- Any thread can access all views by treating it as an STL container

JÜLICH Forschungszentrum

TBB allocators

- Dynamic memory allocation in a multithreaded program must avoid conflicts from `new` calls from different threads
- Global memory lock

TBB allocators

- Interface like `std::allocator`, so that it can be used with STL containers. E.g., `std::vector<T, tbb::cache_aligned_allocator<T>>`
- `tbb::scalable_allocator<T>` : general purpose scalable allocator type, for rapid allocation from multiple threads
- `tbb::cache_aligned_allocator<T>` : Allocates with cache line alignment. As a consequence, objects allocated in different threads are guaranteed to be in different cache lines.

Chapter 5

Lessons from matrix multiplication

Lessons from matrix multiplication

Exercise 6.1:

In the examples folder, you will find a `MatMul` subfolder, containing a written lesson called `SessionMatrix.pdf`. This file contains 8 stages organised as exercises starting with a naive implementation of a matrix type in C++, and ending with something with reasonably respectable performance (comparable to what is possible with, e.g., Eigen, or other BLAS libraries) on a single node on JURECA. It only uses concepts introduced in this course, and does not call any linear algebra library function. Work through the exercises and test the different stages on JURECA!